

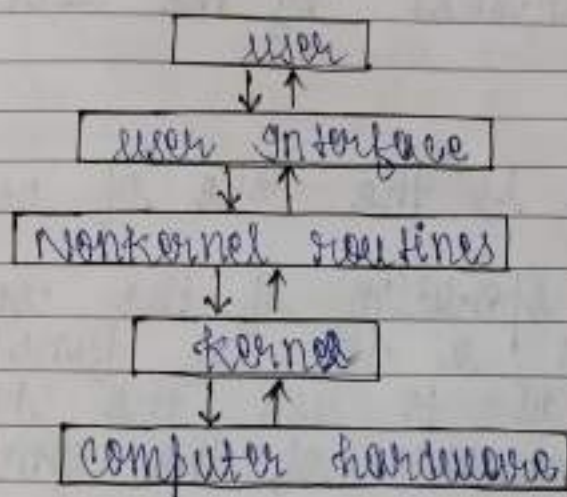
12/04/2024

Chapter - 1

Introduction to operating system

⇒ A individual user has an abstract view of the computer system.

Designer abstract view of O.S :-



• user interface :-

⇒ The user interface accepts commands to execute programs and use resources and services provided by the operating system.

⇒ It is either a command line interface, as in unix or linux, which displays a command prompt to the user and accepts a user command, or is a graphical user interface (GUI), as in the graphical windows operating system, which interprets mouse clicks on icons as user commands.

• Kernel routines :-

- ⇒ These routines implement user commands concerning execution of programs and use of the computer's resources; they are invoked by the user interface.
- kernel :-

⇒ The kernel is the core of OS.

⇒ It controls operation of the computer and provides a set of functions and services to use the CPU, memory, and other resources of the computer.

⇒ The functions and services of the kernel are invoked by the user interface and by user programs.

• operation of an OS :-

⇒ The primary concern of an OS during its operation is execution of programs, use of resources, and prevention of interference with programs and resources.

The main three principles of the OS are as follows :-

(i.) Program management :-

⇒ The OS initiates programs, manages their execution on the CPU, and provides them when they complete their execution.

⇒ Since many programs exist in the system at any time, the OS performs a function called scheduling to select a program for execution.

(ii.) Resource management :-

⇒ The OS allocates resources like memory and I/O devices when a program needs them.

(iii.) Security and protection :-

⇒ The OS implements non-interference in users activities through strict actions of the security and protection functions.

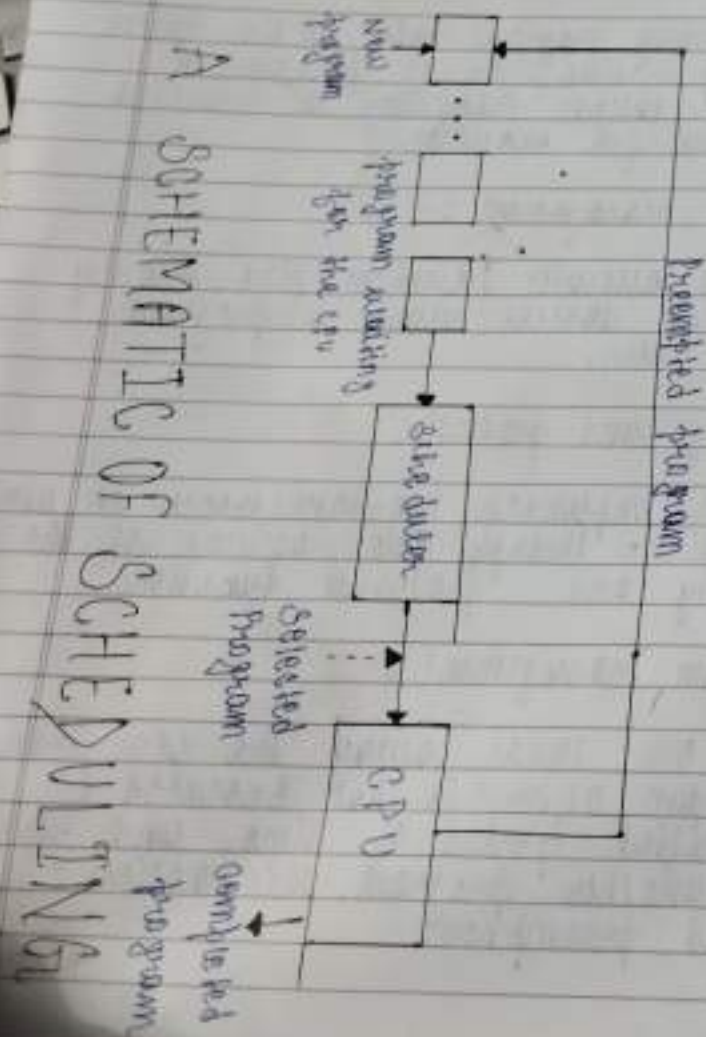
(iv.) Program management :-

⇒ The OS takes away the CPU from a program when it has executed for the specified period of time, and gives it to another program, this action is called preemption.

13/04/24

(vi) Resource Management :-

⇒ The resource allocation and deallocation of the data is done by Resource Management.



Virtual resource :-

⇒ A virtual resource is a fictitious resource, it is an illusion by an OS through use of a real resource.

⇒ Most operating system provide a virtual resource called virtual memory, which is an illusion of a memory that is larger in size than the real memory of a computer.

⇒ It's use enables a programmer to execute a program whose size may exceed the size of real memory.

Security & protection :-

⇒ The security function counters threat of illegal use or interference that are posed by persons or programs outside the control of an operating system.

⇒ The protection function counters similar threats posed by its users.

⇒ The process of computer system to verify the user using password is known as authentication.

⇒ The code Red worm of 2001 spread to a quarter of a million computer systems in 9 hours.

⇒ When a computer system is connected to the internet, and a user downloads a program from the internet, there is a danger that a downloaded program may interfere with other programs or resources in the system. This is a security threat because the interference is caused by some person outside the system, called an intruder. Such threats are imposed through a Trojan horse.

Trojan horse :-

⇒ It is a program that has a known legitimate function and a well-disguised malicious function, or a virus, which is a piece of code with a malicious function that attaches itself to other programs in the system and spreads to other systems when such programs are copied.

⇒ Worms are the another security threats.

Internet firewalls are used from the period of time to remove internet traffic through a computer system.

Processes and threads :-

⇒ A single CPU can have up to 2 threads per core.

Threads :-

⇒ The thread is created by the process.

⇒ The threads are always created by the operating system for performing a task of a specific application.

⇒ It helps the CPU to save time.

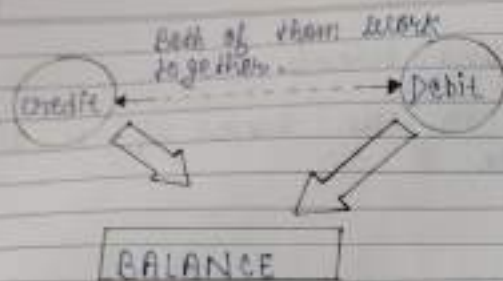
Process :-

⇒ It is an execution of a program.

Process synchronization :-

⇒ Processes that have a common goal must co-ordinate their activities so they can perform their actions in a desired order. This requirement is called process synchronization.

Ex: Credit and Debit balance in bank account.



Message passing :-

⇒ When a process sends some information in a message to another process, the operating system stores the message in its own data structures until the destination process makes a request to receive a message.

Scheduling :-

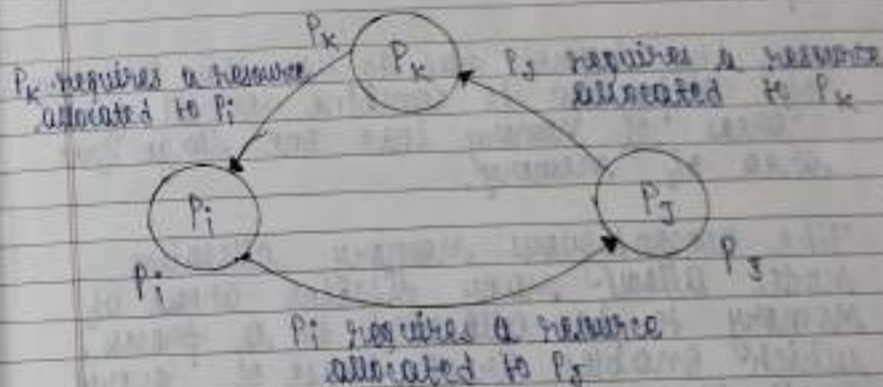
⇒ It is the process through which the CPU is guided by OS to perform resources scheduling process.

Deadlocks :-

⇒ If a resource requested by some process P_i is currently allocated to process P_j , P_i has to wait until P_j releases the resource. Such wait sometimes cause a deadlock.

This is a situation in which the process waits for another process.

A deadlock adversely affects the performance of the system because processes involved in the deadlock cannot make any progress and resources allocated to them are wasted.



☞ Illustration of Deadlock involving three process.

Multiprocessor operating system :-

⇒ A multiprocessor operating system perform effectively because several CPUs are used for fast processing of the resources and scheduling.

Management of memory :-

⇒ In the classical model of memory allocation, a single contiguous area of memory is allocated to a process. This model does not support needs of memory area that is not large enough to accommodate a new process.

⇒ So, the kernel has to use the technique of compaction to combine several free areas of memory into one large free area of memory.

7/05/24 ⇒ The noncontiguous memory allocation model allows many distinct areas of memory to be allocated to a process, which enables direct reuse of several small areas of memory.

Virtual memory :-

⇒ Modern operating system provide virtual memory, which is a storage capability that is larger than the actual memory of a computer system.

⇒ The OS achieves it by storing the code and data of a process on a

disk, and loading only some portions of the code and data in memory. This way, a process can operate even if its size exceeds the size of memory.

⇒ The operating system employs the non-contiguous memory allocation to implement virtual memory.

⇒ During operation of the process, the CPU passes each instruction address or data address used by it to a special hardware unit called the memory management unit (MMU).

⇒ If the required instruction or data doesn't exist in memory, the (MMU) causes a "missing from memory" interrupt.

⇒ The OS now loads the portions that contains the required instruction or data in memory.

⇒ "missing from memory" process slow down the process of execution of the programs.

Management of memory :-

⇒ In the classical model of memory allocation, a single contiguous area of memory is allocated to a process. This model does not support needs of memory area that is not large enough to accommodate a new process.

⇒ So, the kernel has to use the technique of compaction to combine several free areas of memory into one large free area of memory.

⇒ The noncontiguous memory allocation model allows many distinct areas of memory to be allocated to a process, which enables direct reuse of several small areas of memory.

Virtual Memory :-

⇒ Modern operating system provide virtual memory, which is a storage capability that is larger than the actual memory of a computer system.

⇒ The OS achieves it by storing the code and data of a process on a

disk, and loading only some portions of the code and data in memory. This way, a process can operate even if its size exceeds the size of memory.

⇒ The operating system employs the non-contiguous memory allocation to implement virtual memory.

⇒ During operation of the process, the CPU passes each instruction address or data address used by it to a special hardware unit called the memory management unit (MMU).

⇒ If the required instruction or data doesn't exist in memory, the MMU causes a "missing from memory" interrupt.

⇒ The OS now loads the portions that contains the required instruction or data in memory.

⇒ "Missing from memory" process slow down the process of execution of the program.



Management of files and I/O devices :-

⇒ A file system acts as a layered organization to schedule the various layers involved in fulfilling these expectations.

⇒ The upper layer, which is the file system itself, permits a user to share his files with some other users, implements file protection and provides reliability.

⇒ To implement an operation on a file, the file system invokes the lower level, which contains the input output control system [IOCS].

⇒ This layer ensures fast access to files by a process, and efficient use of I/O devices.

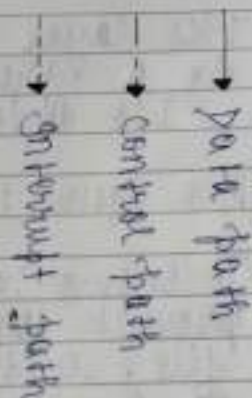
(i.) File system :-

⇒ To ensure reliability, the file system prevents damage to the data in a file, and to its own data such as a directory, which is called the "Metadata".

Distributed operating systems :-

⇒ The distributed operating system provides three key advantages :-

- speeding up of a computer application by scheduling its processes in different nodes of the system simultaneously.
- High reliability through redundancy of computer systems and their resources.
- Resource sharing across node boundaries.

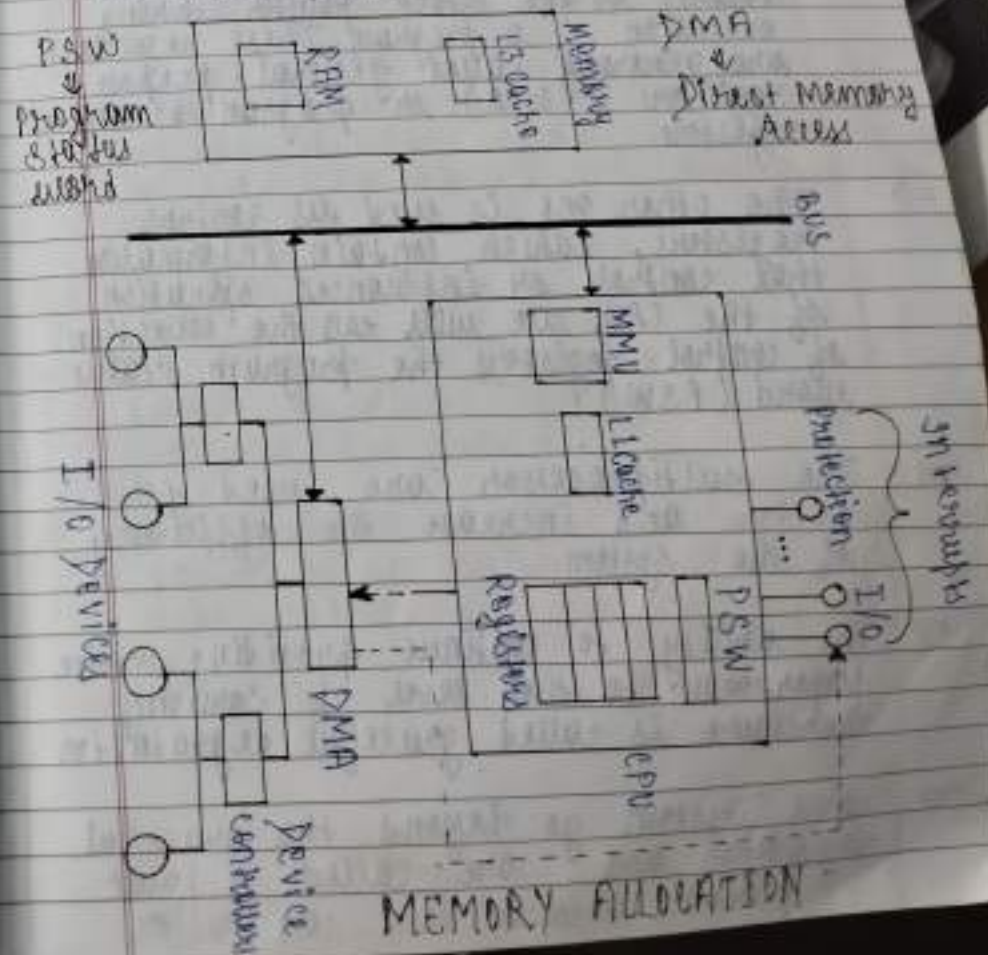


Chapter - 2

The OS, the computer, and user programs.

Fundamental principles of OS operation :-

⇒ The kernel exists in memory taking operation of the OS, and executes instructions on the CPU to implement its control functions and services.



THE CPU :-

⇒ Two features of CPU are visible to user programs in the operating system.

⇒ The first is those registers that are used to hold data, addresses, index values, or the stack pointer during execution of a program. These registers are generally called general-purpose registers (GPR) or program-accessible registers.

⇒ The other one is used as control registers, which contain information that controls or influences operation of the CPU; we will call the collection of control registers the program status word (PSW).

⇒ The multiprocessor are used for better and increase the efficiency of the system.

⇒ The ability to continue providing service proportional to the level of surviving hardware is called graceful degradation.

Some systems go beyond the graceful degradation and are called fault

a failure of any single component and still continue operation.

⇒ The multiple-processor system is of two types :-

(i) Asymmetric multiprocessing :-

⇒ Each processor is assigned a specific task.

⇒ A bus processor controls the system, the other processor either look to the bus for instruction or have predefined tasks.

(ii) Symmetric multiprocessing [SMP] :-

⇒ Each processor performs all tasks within the operating system, all processors are peers.

⇒ Each processor has its own set of registers, as well as private and local cache.

PROCESS MANAGEMENT

② The more complex the operating system is, the more it is expected to do on behalf of its users.

The process :-

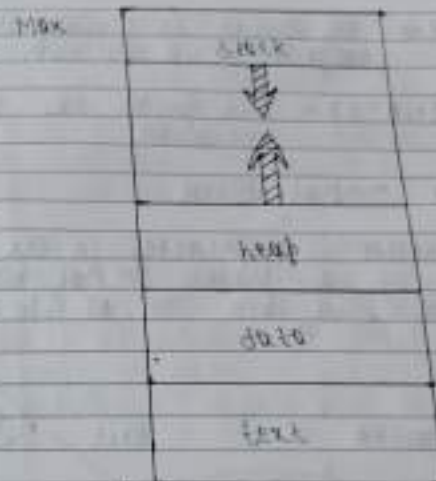
⇒ A process is more than the program code, which is sometimes known as the text section.

⇒ A process generally includes the process stack, which contains temporary data such as function parameters, return addresses, and local variables, and a data section, which contains global variables.

⇒ A process may include a heap, which is memory that is dynamically allocated during program run time.

⇒ A process is an active entity, with a program counter specifying the next instruction to execute and a set of associated resources.

⇒ A program becomes a process when an executable file is loaded into memory.



Process in Memory

Process State :-

⇒ As a process executes, it changes state.

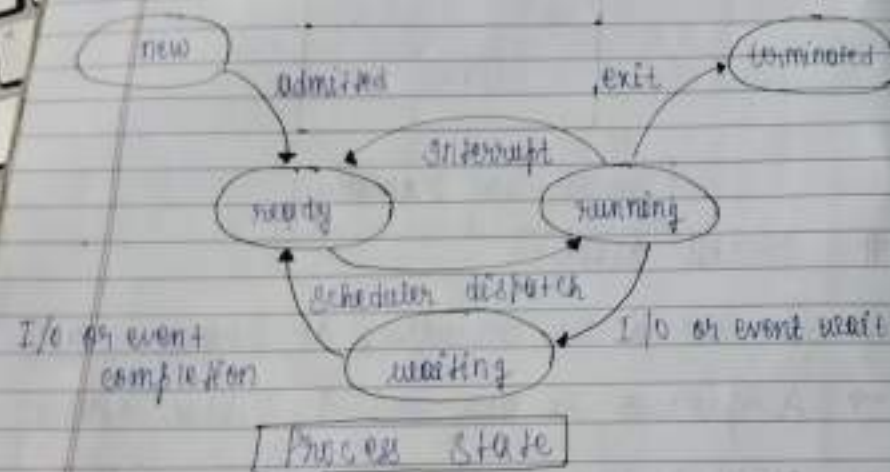
⇒ A process is executed by these states:-

- NEW ⇒ The process is being created.
- RUNNING ⇒ Instructions are being executed.
- WAITING ⇒ The process is waiting for some event to occur (such as I/O completion).

- **READY** → The process is waiting to be assigned to a processor.
- **TERMINATED** → The process has finished execution.

Process control Block :-

⇒ Each process is represented in the operating system by a process control block (PCB) also called a task control block.



⇒ A PCB contains many information in a specific process such as :-

- **PROCESS STATE** → The state should be new, ready, running, waiting, halted and so on.

- **PROGRAM COUNTER** → The counter indicates the address of the next instruction to be executed for this process.

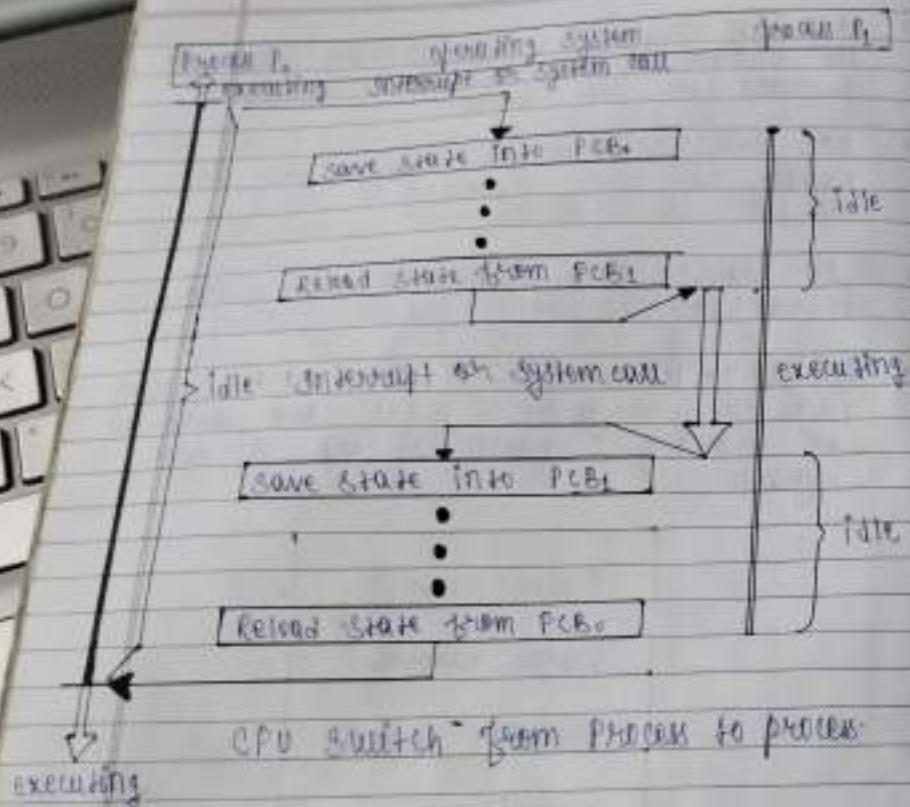
- **CPU REGISTERS** → The registers vary in number and type depending on the computer architecture.

- **CPU-SCHEDULING INFORMATION** → The information includes process priority.

- **MEMORY-MANAGEMENT INFORMATION** → Registers, page tables, etc. are used as an memory system of the operating system.

Process state
Process number
Program counter
registers
memory limit
list of open files
...

Process control block (PCB)



THREADS:-

⇒ When a process is running a word-processor program, a single thread of instructions is being executed.

⇒ This single thread of control allows

the process to perform only one task at a time.

⇒ The more number of threads in the case of the system helps run and execute more number of program.

20/02/24

Operating System

Types of OS

- Batch OS → 1960 Punchcard or Magnetic tape
- Multiprogramming → Non-preemptive
- Realtime → preemptive
- Distributed
- Embedded

- * Multiprogramming → in RAM. Just allocate the job in a order that OS can easily work upon it.
- * Throughput - Number of jobs executed per unit time

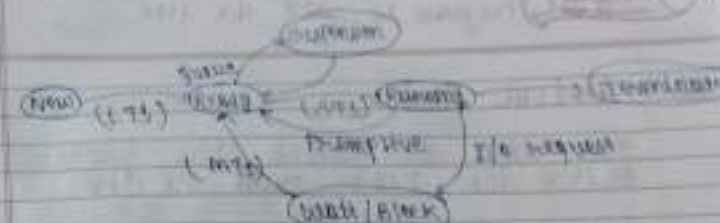
(i) System call

(ii) User Mode Kernel Mode

(iii) Process State



The process in which kernel mode change then this process is known as system call. Multiprocessing



- LTS → Long term Scheduling
- STS → Short term Scheduling
- MTS → Middle term Scheduling
- Realtime OS → Hard Realtime, Soft Realtime

20/02/24

CPU Scheduling

preemptive

non-preemptive

(i) SRTF (Shortest remaining time first)

(i) FCFS (First come first serve)

(ii) LRTF (Longest remaining time first)

(ii) SJF (Shortest job first)

(iii) LTF (Longest job first)

(iii) Round Robin (R-R)

(iv) priority based

(iv) HRRN (Highest response ratio next) priority

- ④ If same arrival time come for 6 jobs then program id will be same.

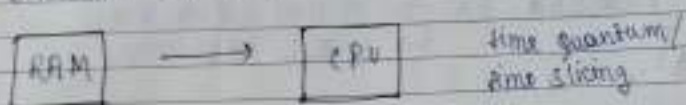
(i) Arrival time :-

The time at which the process enters the ready queue or state.



(ii) Burst time :-

Time required by a process to get executed on CPU.



Duration of time in CPU

(iii) completion time :-

The time at which process complete its execution.

point of time.

(iv) Turn around time = (TAT).

(completion time - Arrival time)

initial time to the completion time in CPU

non-preemptive = All the work is to be done and then finish them

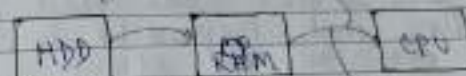
preemptive = All the work is to be done for all process individually for everyone will get time quantum

(v) waiting time :-

$$TAT - RT$$

(vi) Response time :-

The time at which a process get CPU for first time - (Arrival time)



Round time is seen that

FCFS Algorithm (non-preemptive)

$$WT = R.T$$

Process	Arrival time	Burst time	Completion time	TAT	WT	RT
P ₁	0	2	2	2	0	0
P ₂	1	2	4	3	1	1
P ₃	5	3	8	3	0	0
P ₄	6	4	10	4	2	2

$$\text{Avg TAT} = \frac{14}{4} = 3.5$$

using GANTT chart :-



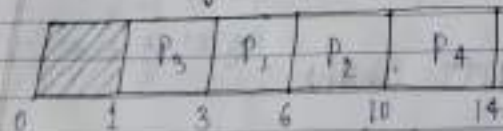
$$\text{Response time} = \begin{matrix} 0-0=0 \\ 2-1=1 \\ 8-5=3 \\ 10-6=4 \end{matrix}$$

SJF Algorithm :-
→ Shortest Job first

- If the burst time come similar to the both process then the arrival time is seen i.e. whose arrival time is less that process will start.
- If both arrival time and burst time come similar then program id is seen for the execution by CPU.

Process	AT	BT	CT	TAT	WT	RT
P ₁	1	3	8.6	5	2	8.2
P ₂	2	4	10	8	4	12
P ₃	1	2	3	2	0	2.0
P ₄	4	4	14	10	6	18

using GANTT chart



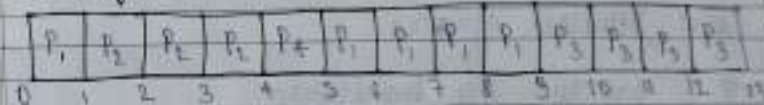
If SJF given in preemptive approach to solve then we can say it can be RTF.

05/08/24

Shortest remaining time first (SRTF) :-
(same as SJF + preemptive)

Process	AT	BT	CT	TAT	WT	RT
P ₁	0	5	8.8	5	4	0
P ₂	1	3	9.2	5	0	0
P ₃	2	4	12	11	7	3
P ₄	4	1	5	1	0	0

using Gantt + chart



process Management :-

process :-
IPC → Inter process communication.
PCB → process control block.

↓
process state

• number
" counter

Register

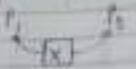
memory limit

Scheduling algorithm

Process is a single program execution.

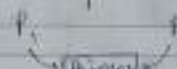
process synchronization

competitive



Race condition

cooperative



eg → producer consumer problem

problem occurs in process synchronization:

(i) critical section problem.

(ii) Race condition.

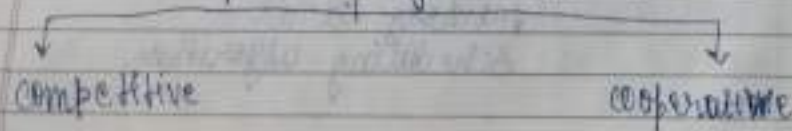
(iii) deadlock problem.

process synchronization

⇒ Interprocess communication leads to the need of synchronization as many processes communicate with each other for sharing say variables, registers, memory, etc.

The two basic environments encountered during synchronization are:-

process synchronization



Apart from these two environments there also exist another environment that is called independent environment where no two processes share any kind of information among them and hence, they don't need synchronization.

Q) what if synchronization is not performed

If synchronization is not performed the system have to face conditions like:-

(i) critical section problem

(ii) Race condition.

(iii) deadlock

(i) critical section problem:-



Q) what is critical section?

⇒ It is the part of the program that access shared resources.

Q) what is competitive synchronization?

⇒ one or more processes are said to be in competitive synchronization if they compete for accessibility of a shared resource.

Q) what is cooperative synchronization?

⇒ If one or more processes are said to be in cooperative synchronization if one process affects the working of another process then it is called cooperative synchronization.

Q) what is Race condition?

⇒ Race condition is a condition which occurs due to competitive synchronization. It is a condition where two or more processes are sharing the same resource for example say variable. then the competition among the processes to access the variable if not synchronized leads to race condition.

Ex: for example, if we have two processes say P_1 and P_2 sharing a common variable say, (num) initialized with value 5 i.e. $num = 5$, now P_1 wants to

increment the value of (num) by 1 and P_2 wants to decrement the value of (num) by 1. so

If P_1 and P_2 were synchronized the output could have remained unchanged i.e. num would have same value 5. but if the processes are not synchronized either P_1 or P_2 would have and the output would be either 6 or 4, leading to race condition.

Q) what is critical section problem?

⇒ critical section refers to the part of a program that occurs to access shared resource.

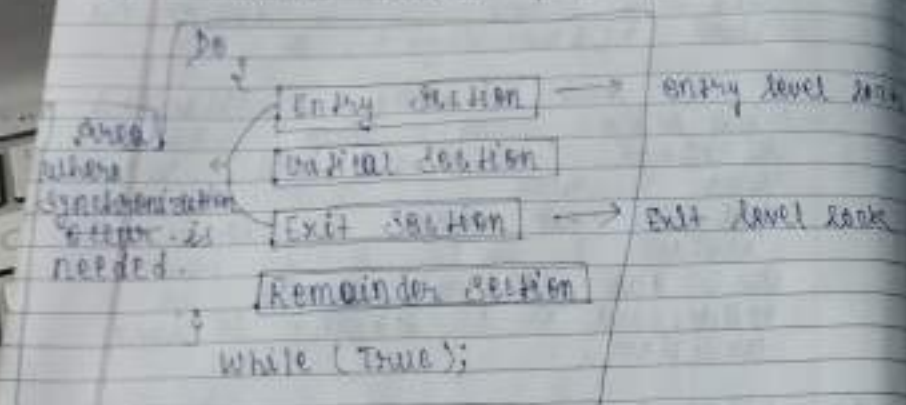
Example of Race condition.

Prog	
P_1	P_2
int num = 5;	int num = 5;
// start	// start
int x;	int y;
num = x;	num = y;
x++;	y--;
sleep(1);	sleep(1);
return x;	return y;
output:-	

is the value critical section because want number

Ques 1

Critical section problem:-



Ans 2

Critical section problem:-

Solution:-

(i.) Mutual exclusion

⇒ No. 2 process can enter its critical section (cs) at the same time. depends upon O.S.

(ii.) progress

(iii.) Bounded wait.

one process can be taken a specific period of time then the process

Algorithm is machine independent.

comes out from CPU.

⇒ The critical section problem is used to design a set of protocols which ensure that the race condition among the processes will never arise.

⇒ Critical section problem and race condition belongs to competitive synchronization.

count = 1	
P ₁	P ₂
int x, y	int p, q
x = count	p = count
x++;	p--;
return x;	return p;

Shared resource.

cooperative environment

producer consumer problem.

✓ solution is semaphore.

In order to synchronized, the competitive processes, each main task is to solve the critical

section problem. we need to provide a solution in such a way that the following conditions can be satisfied.

Requirement of synchronization
Mechanism (critical section problem)

(i) Mutual exclusion:- The

⇒ The solution must provide mutual exclusion. By mutual exclusion we mean that if one process is executing in critical section then no other process must enter its critical section.

(ii) Progress:-

⇒ progress means that, if one process has completed its critical section then it must release the resource held and should not stop other process to enter its critical section.

(iii) Bounded waiting:-

⇒ operating system must ensure that any process must not be endlessly

waiting for getting into critical section. In this case, it allocates a waiting time bound for every process when it enters its critical section. the process is bound to complete its execution within the bounded time.

notes

functionality of OS

(i) Allocates resource in a client and server level. (multiple user).

(ii) Process management & can be executed with the help of CPU scheduling.

(iii) Storage management can be done through file system to store permanent data.

(iv) Memory management (RAM), swapping of allocation and deallocation of process.

(v) security and privacy.

The interfere between the process can not be done between two or more process, the OS blocks that process, if done.

If P_1 wants to access P_2 data then it cannot because the resource allocation is done among P_1 and P_2 .

Types of OS

(1) Batch :-

Working of Batch

$P_1 = Job_1$
 $P_2 = Job_2$
 $P_3 = Job_3$

For storing data punch cards were used, as well as magnetic tapes, paper tape in 1960s.

The data collected in punch card are punched in the system through the help of operator in the form of Batches.

Punch card

paper tape

Magnetic tape

operator →

another work

Batches

B_1 | B_2 | B_3
3 | 4

similar work →

↓ CPU idle.

I/O

⇒ The B_1 block is allocated in CPU for I/O hardware process then CPU becomes idle. because it is a non-preemptive approach.

Modified version was fortran, and IBM's 709X, in 1960 was used as a monitor for taking data without the help of seeing operator.

(2) Multiprogramming :-

⇒ It is a non-preemptive type approach.

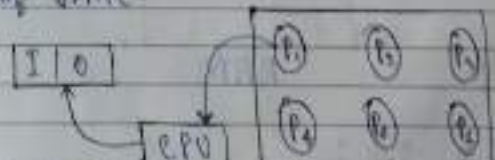
⇒ It allows the processes sequentially in RAM.

Let us take an example :-

If a teacher wants to solve 5 questions of a class having 50 students then he/she has to solve 5 questions of each student one by one because it is a non-preemptive approach.

It is a time consuming approach.

If in solving no 3 question of a student he/she wants to go to bathroom then the teacher will go to another student to solve the question, it helps less consumption of time.



If the process wants I/O processing in between of program then RAM. The CPU will shift to another process for maintaining non-idle non.

(iii) Multitasking / Time sharing os:-

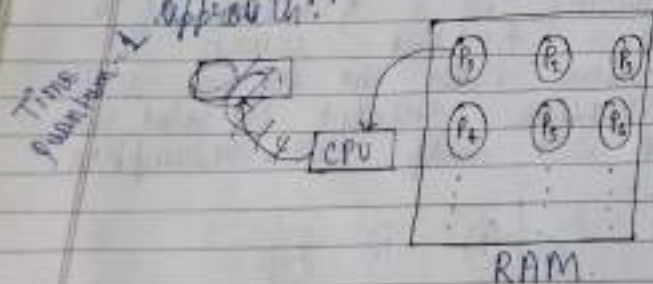
⇒ It is a preemptive approach.

⇒ In this type of os each process get resource in less period of time than a Multiprogramming.

• Let us take an example.

• If a teacher wants to solve 5 question of a class having 60 students, then the teacher solve 2 question of each student one by one because it is a preemptive approach.

This approach is an responsive approach.



(iv) Real time os:-

⇒ The device which depends upon the time for the execution of the process.

(a) Hard real time os:-

⇒ The device which mainly depends upon the time for execution of the process.

Ex: Airplane simulation, missile launching, etc.

(b) Soft Real time os:-

⇒ It also depends upon time but executes after a specific allocated period of time.

(c) Live streaming in youtube, the host says something is delayed by 5 or after five second because of transmission.

(v) Distributed os:- / Types: (i) client server (ii) peer to peer.

⇒ A large number of systems are connected with each other with the help of internet.

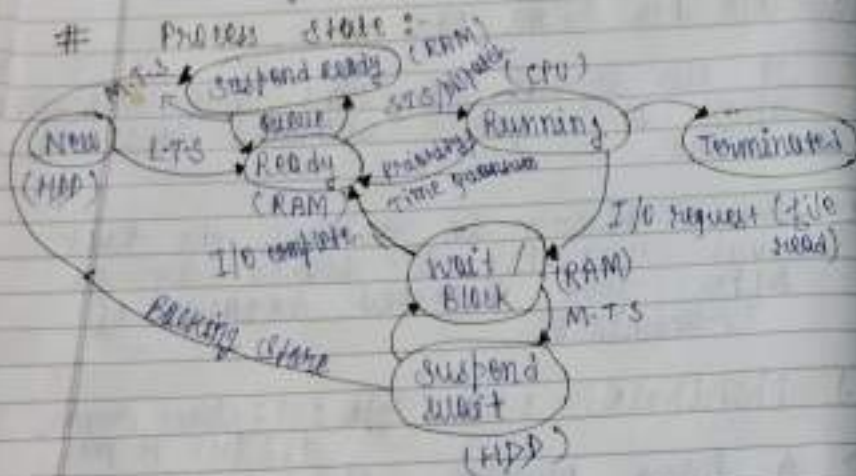
(vi) clustered os:-

⇒ A local area network is converted into a distributed type for connecting system for fast data transfer and execution.

CPU execution speed is in millionth of a second.

(NA) Embedded OS: → microcontroller is used
 ⇒ This type of OS can be used for one type of working.
 Ex: ECU [Engine control unit], Washing machine.

Process state:



LTS → Long term scheduler.

STS → Short term scheduler.

MTS → Middle term scheduler.

← preemptive in between Ready queue and Running.

← ^x Non-preemptive in between Ready queue and Running.

fork(): system call:

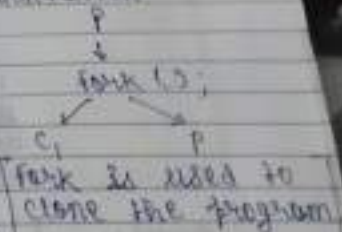
fork() → 0 (child)
 → +1 (parent)
 → -1 (child not created)

Just for representation.

include <stdio.h>
 int main()

```

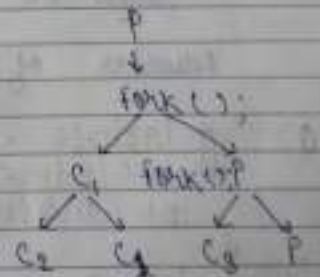
{
    fork();
    printf("Hello world");
    return 0;
}
    
```



include <stdio.h>
 int main()

```

{
    fork();
    fork();
    printf("Hello world");
    return 0;
}
    
```



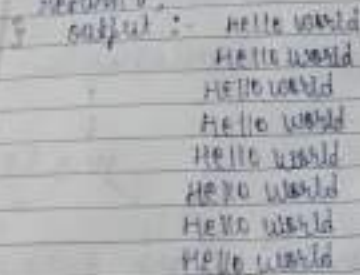
output:
 Hello world
 Hello world
 Hello world
 Hello world

include <stdio.h>
 int main()

```

{
    
```


L



71

Number of child = $[8 - 1]$

95

4

How I
output

LA 2

using formula $a = 2^n$

There is 4 horizontal lines with number 1 to 4 below.

$$= 2^8$$

$= 16$

Child.

$$2^n - 1$$

15

92



44

19

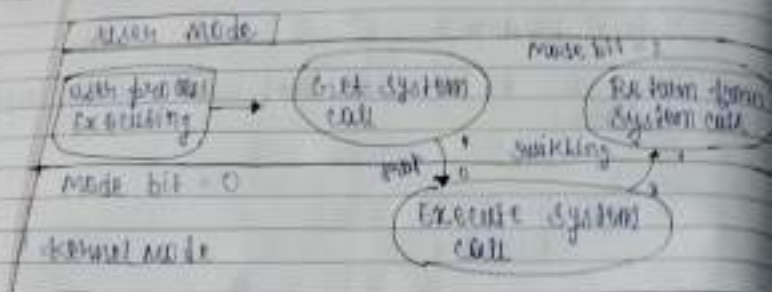
⑧

25

Q

8

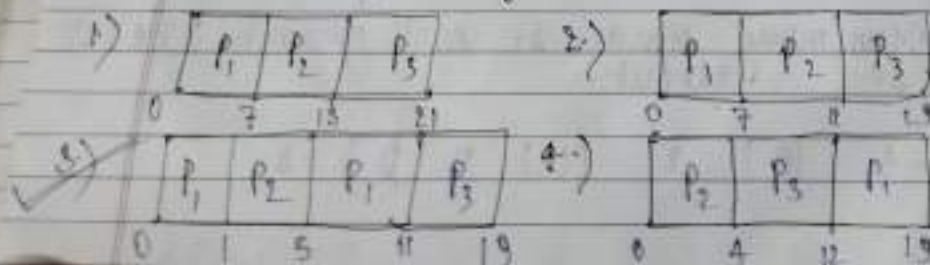
User Mode vs Kernel Mode



Shortest Job First: preemptive approach.

Process	AT	BT
P ₁	0	7
P ₂	1	4
P ₃	2	8

The Gantt chart for preemptive SJF scheduling algorithm is

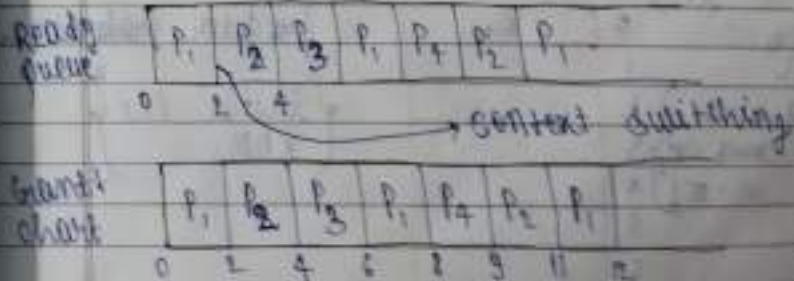


using Gantt chart



Round Robin preemptive approach: Time quantum = 2 units

Process	AT	BT	CT	TAT	WT	RT
P ₁	0	5	12	12	7	0
P ₂	1	4	11	10	6	1
P ₃	2	2	6	4	2	2
P ₄	4	1	9	5	4	4



for time quantum = 3 units

Process	AT	BT	CT	TRT	WT	RT
P ₁	0	5	10	10	5	0
P ₂	1	4	12	11	7	4
P ₃	2	2	8	6	4	1
P ₄	4	1	11	7	6	6

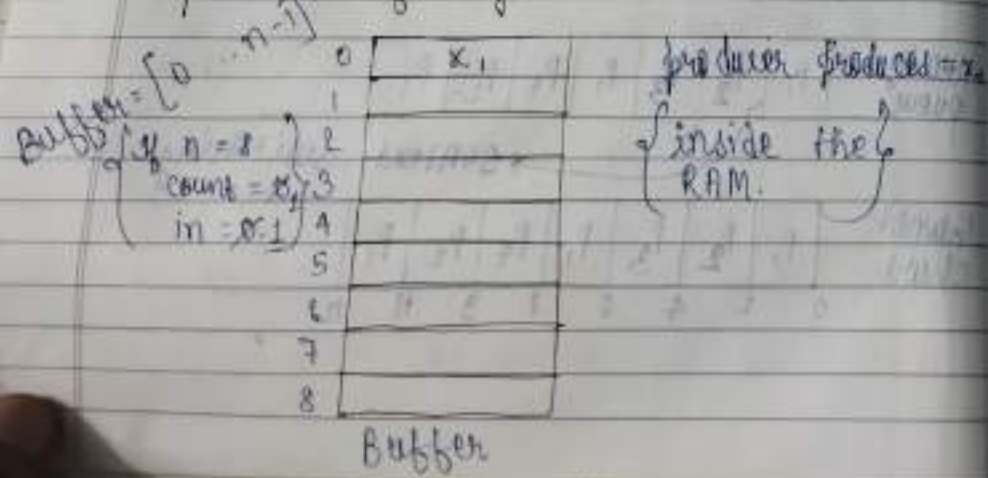
Ready queue

P ₁	P ₂	P ₃	P ₁	P ₄	P ₂
----------------	----------------	----------------	----------------	----------------	----------------

Running queue

P ₁	P ₂	P ₃	P ₁	P ₄	P ₂
----------------	----------------	----------------	----------------	----------------	----------------

Producer consumer problem / classical problem of synchronization:-



for non preemptive approach to problem occur.

Producer preemptive approach creates a large number of process.

```

int count = 0;
void producer(void) {
    int item;
    while (true) {
        producer_item(item);
        while (count == n) {
            // Buffer full condition
        }
        Buffer[in] = item;
        in = (in + 1) % n;
        count = count + 1;
    }
}

// consumer

```

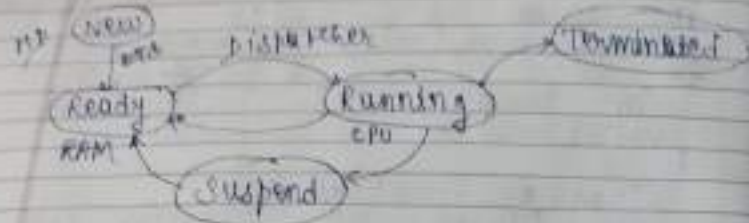
```

void consumer(void) {
    int itemc;
    while (true) {
        while (count == 0) {
            // Buffer empty condition
        }
        itemc = Buffer[out];
        out = (out + 1) % n;
        count = count - 1;
        process_item(itemc);
    }
}

```

02/12/24

Process State Transition Diagram:-



ROM → Bootstrap program for booting.

Q) List the differences between process and program.

Program	Process
---------	---------

IDE → Integrated development environment.

⊗ Source code $\xrightarrow{\text{compiler}}$ Object code $\xrightarrow{\text{linker}}$ exe
 loading $\xrightarrow{\text{New/Ready}}$ Dispatching $\xrightarrow{\text{CPU}}$ output

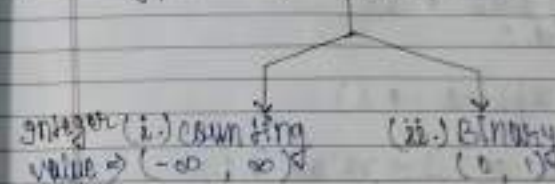
02/12/24

It is not a parameter or path type variable.

SEMAPHORE :-

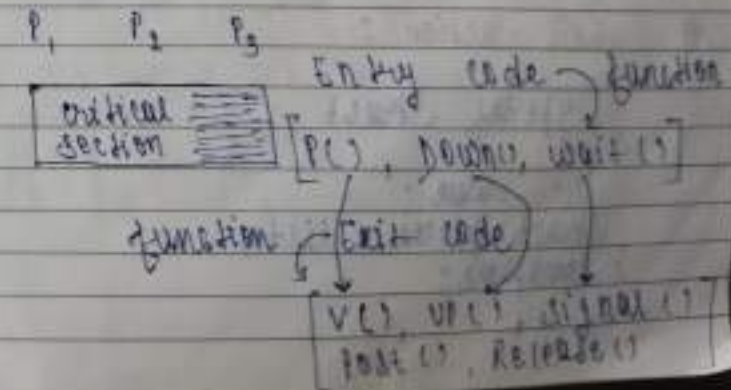
⇒ Semaphore is an integer variable, which is used in mutual exclusive manner by various concurrent cooperative processes in order to achieve synchronization.

Types of semaphore



(i) counting :-

For process to be executed inside the critical section, it should follow entry code as well as exit code.



Entry section :-

```
down (semaphore *s)
```

```
{ s->value = s->value - 1;
```

```
typedef struct { s=1
```

```
int value;
```

```
struct process *list;
```

```
}; semaphore;
```

```
down (semaphore *s)
```

```
{ s->value = s->value - 1;
```

```
if (s->value < 0)
```

```
{ put process (P) in suspended
```

```
(s->list) list;
```

```
block ();
```

```
}
```

```
}
```

Exit section :-

```
typedef struct
```

```
{ int value;
```

```
struct process *list;
```

```
}; semaphore;
```

```
up (semaphore *s)
```

```
{ s->value = s->value + 1;
```

```
if (s->value <= 0)
```

```
{ select a process from suspended list
```

```
and ./ s->list;
```

```
wakeup (P);
```

```
}
```

Ex:-

3 → three process

It will go from entry section to exit section

Entry section

Step 1)

2 → process (P₁) enters the CS

Step 2)

1 → process (P₂) enters the CS

Step 3)

0 → process (P₃) enters the CS

Step 4)

-1 → process (P₄) does not enter the CS

Step 5)

-2 → process (P₅) does not enter the CS

Now if process P_1 wants to exit from the CS then

Step 6) $\begin{matrix} 3 \\ -1 \end{matrix} \rightarrow P_1$ comes out of the CS

Step 7) $\begin{matrix} 3 \\ 0 \end{matrix} \rightarrow P_2$ comes out of the CS

Step 8) $\begin{matrix} 3 \\ 1 \end{matrix} \rightarrow P_3$ comes out of the CS

Now process P_4 can go into the CS

Step 9) $\begin{matrix} 3 \\ 0 \end{matrix} \rightarrow P_4$ enters CS

Now, if process P_4 wants to exit from the CS then

Step 10) $\begin{matrix} 3 \\ 1 \end{matrix} \rightarrow P_4$ comes out of the CS

Now, process P_5 can go into the CS

Step 11) $\begin{matrix} 3 \\ 0 \end{matrix} \rightarrow P_5$ enters the CS

And so, on.

As the process enters one by one in the CS as a multiprogramming system but to know we just illustrate with an example, shown previously

(ii) Binary :- {0, 1}

(a.) Entry Section :-

down (semaphore s)

typedef struct

{
int value;
struct process * list;
} semaphore;

down (semaphore * s)

if (s->value == 0)

{

s->value = 0;

}

else

{

put process (PCB) in suspended list

s->list;

block();

}

Exit decision

externally struct

int value;

struct process * list;

semaphore;

up (semaphore * s)

if (suspended list / s → list is empty)

// s = 0 1, s = 0, s = 1

s → value = 1;

else

Select a process from suspended list

s → ssc and

wakeup (s);

Ex → let P_1 and P_2 be the process

let $s = 0$

then neither P_1 nor P_2 process can execute the CS because entry section code will be executed and the code will put both the process P_1, P_2 in suspended list.

s = 0

let $s = 1$

P_1

down (s)

$s = 0$
[P₁] → [CS]

up (s) → s = 1

P_2

down (s)

$s = 0$
[P₂] → [CS]

up (s) → s = 1

As P_1 enters the CS the value of $s = 0$ get mutual exclusion.

As P_2 enters the CS the value becomes 0 and so on.

solution of producer - consumer problem

15/02/24

consumer

empty = 8

full = 8

down (full);

down (s);

item = Buffer [out];

out = (out + 1) mod n;

up (s);

up (empty);

Producer

Producer Item (item);

down (empty);

down (s);

Buffer [in] = item;

0	a	initial section
1	b	
2	c	
3	d	
4		
5		
6		
7		
8		

Buffer

empty = 8

full = 8

s = 0

$In = (In + 1) \bmod N$
 $if (In == 0)$
 $if (full == 0)$

Semaphore

Binary

$S = 1 \text{ or } 0$

counting

$full = 0$ number of filled slots
 $empty = N$ number of empty slots

Threads :-

- ⇒ A thread is a single sequence stream within a process.
- ⇒ Threads are also called lightweight processes as they possess some of the properties of processes.
- ⇒ Each thread belongs to exactly one process.
- ⇒ In an OS that supports multithreading, the process can consist of many threads but threads can be effective only if the CPU is more than 1 otherwise.

two threads have to context switch to that single slot.

Q) what is thread in operating system?

⇒ In a process, a thread refers to a single sequential activity being executed. These activities are also known as thread of execution or thread control.

Q) Why do we need thread?

- ⇒ (i) Threads run in parallel improving the application performance.
- (ii) Each such thread has its own CPU state and stack, but they share the address space of the process and the environment.
- (iii) Threads can share common data so they do not need to use inter-process communication.
- (iv) Like the processes, threads also have states like ready, executing, blocked etc.

(vi) priority can be assigned to the threads just like the process, and the highest priority thread is scheduled first.

(vii) Each thread has its own Thread Control Block (TCB), same as [PCB]

(viii) like the process, a context switch occurs for the thread, and register contents are saved in (TCB).

(ix) As threads share the same address space and resources, synchronization is also required for the various activities of the thread.

27/08/24

Thread :-

Q.) what is thread?

⇒ Threads are small components of a process and multiple threads can run concurrently and share the same code, memory, variables, etc.

Each thread shares the same code, data and heap blocks but will have its own stack. Threads are often called lightweight processes because they have their own

stack of memory. Multiple threads are run together through the CPU functionality known as multithreading.

Process

Thread

(i.) A process is an instance of a program that is being executed in process.

(ii) processes are independent of each other and hence don't share memory or other resource.

(iii) each process is treated as a new process by the OS.

(iv) context switching between two processes takes much time as they are heavy compared to threads.

(v) The OS takes more time to terminate a process.

Thread is a segment of a process in a light weight process that is managed by a scheduler. i.e. independently

Threads are interdependent and share memory.

(vi) The OS takes all the user level threads as a single process.

(vii) context switching between the threads is fast because they are light weight.

(viii) Threads can be terminated in very little time.

(Ans) New process creation from thread needs less time as it does not take time for creation as each new thread takes all the resources.

fork() system call:-

⇒ The fork system call is used in OS to create a new process by duplicating the calling process.

⇒ The new process is an exact copy of the parent process, with its own address space and memory.

priority based scheduling:-

⑧ preemptive approach:-

Priority	Process	A.T	B.T	C.T	TAT	WT	RT
10	P ₁	0	5	12	12	7	0
20	P ₂	1	4	8	7	3	0
30	P ₃	2	2	4	2	0	0
40	P ₄	4	1	5	1	0	0

Higher the number, higher the priority

Method: priority

Mode: preemptive

using Gantt chart

P ₁	P ₂	P ₃	P ₃	P ₄	P ₂	P ₂	P ₂	P ₁	P ₁	P ₁	P ₁
0	1	2	3	4	5	6	7	8	9	10	11

20/10/20

Deadlock:-

1) What is a deadlock?

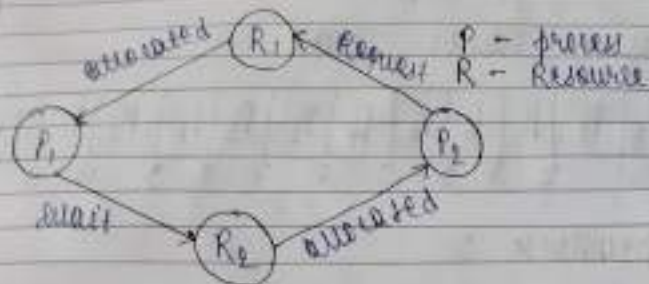
2) What is the necessary condition for deadlock?

3) What are the methods to prevent deadlock?

4) What are the methods to avoid deadlocks?

1) All the processes in a system require some resources such as CPU, file storage, I/O devices, etc to execute it. Once the execution is finished the process releases the resource it was holding.

However when many processes run on a system they also compete for their resources and this may lead to deadlock situation



V.V.V.

2) Necessary condition for deadlock?

(i.) Mutual exclusion

(ii.) No preemption

(iii.) Hold and wait

(iv.) circular wait / bounded wait

Resource Allocation graph (RAG):-

Vertex

Process vertex

Resource vertex

• for availability of resources

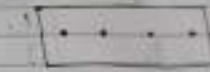
Resource vertex

single instance



Ex -> CPU, Monitor

multiple instance



Ex -> Pages

Edge

Assign edge

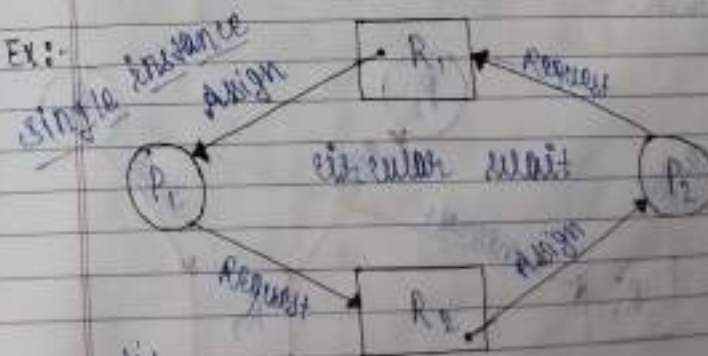


P - process
R - Resource

Request edge



Ex:-



cycle

To know is it in deadlock condition
 way to know about deadlock or
 not.

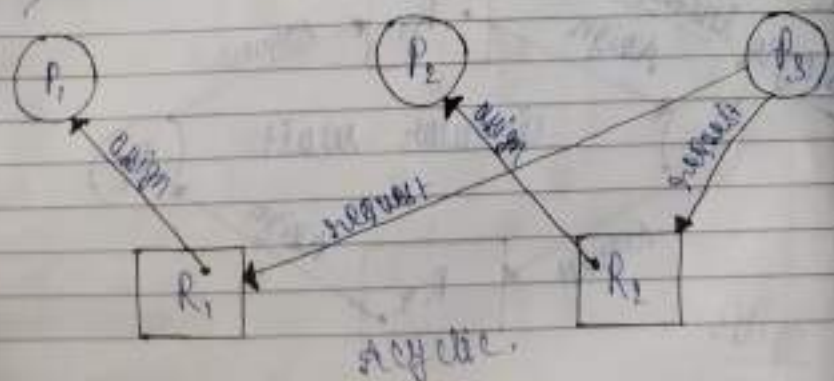
P.	Allocate		Request	
	R_1	R_2	R_1	R_2
P_1	1	0	0	1
P_2	0	1	1	0

Allocate = 1
 Non-Allocation = 0.

Availability of resource for P_1 & P_2
 to be executed = $(0, 0)$
 (R_1, R_2)

Here, neither process P_1 nor process
 P_2 can be executed. because both
 the process request resource that are
 with them that is held and wait
 process is, the process comprises
 deadlock.

Ex:



To know is it in deadlock or not

	Allocate		Request	
	R_1	R_2	R_1	R_2
P_1	1	0	0	0
P_2	0	1	0	0
P_3	0	0	1	1

Availability of resource = $(0, 0)$
 (R_1, R_2)

Here, the process (P_1) can execute
 because no request has to be done
 (R_1, R_2)

Availability of resource = $(1, 0)$
 after execution of process (P_1)

Now, process (P_2) can also be executed
 because it also doesn't require
 any request.
 $(0, 0)$

Availability of resource = $(1, 1)$
 after execution of process (P_2)

Now, process (P_3) can be executed
 because we have the availability
 of resource for the execution of
 the process (P_3).

single instance $\rightarrow$ allocation of one resource.
Ex $\rightarrow$ CPU, monitor.

Note :-

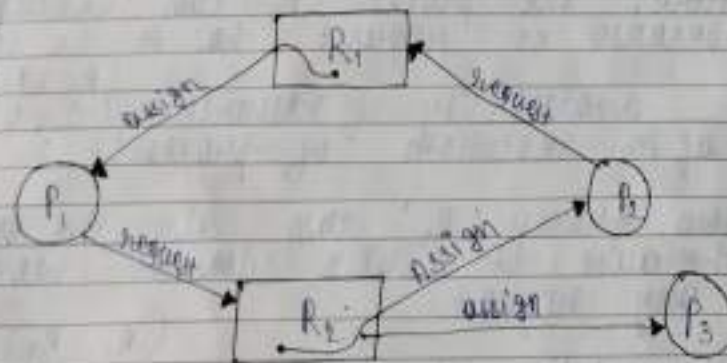
(i) In an single instance resource, if RAB has circular wait (cycle)

$\Rightarrow$ it always creates deadlock

(ii) In an single instance, if RAB has no cycle then no deadlock.

Multiple instance :-

Ex $\rightarrow$



	Allocation		Request		
	R_1	R_2	R_1	R_2	
P_1	1	0	0	1	
P_2	0	1	1	0	
P_3	0	1	0	0	

Availability of resource = $(0, 0)$

Here, at this instance process (P_2) will be executed.

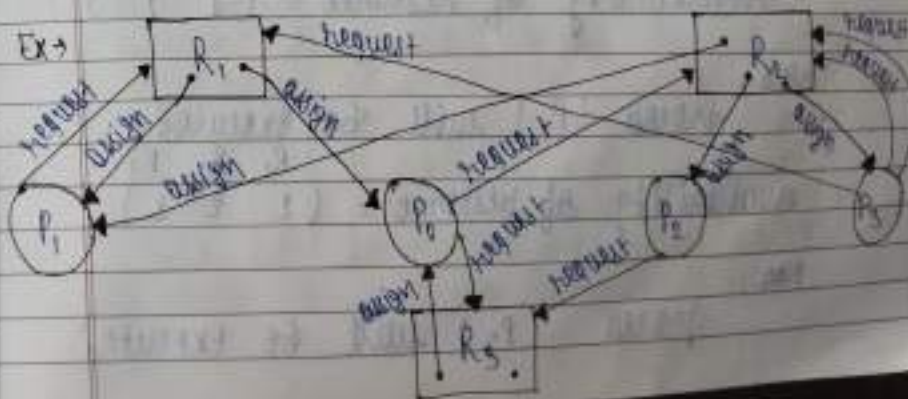
Availability of resource = $(0, 1)$, when process (P_2) is terminated.

Now, process (P_1) can execute, because it wants resource (R_2) for execution and we have the availability of resource.

Availability of resource = $(1, 1)$, when process (P_1) is terminated.

Now, process (P_3) can execute with the help of resource (R_1).

Hence this process does not have deadlock situation.



→ for availability of resource.

	allocate			request		
	R_1	R_2	R_3	R_1	R_2	R_3
P_0	1	0	1	0	1	1
P_1	1	1	0	1	0	0
P_2	0	1	0	0	0	1
P_3	0	1	0	1	2	0

$$\text{Availability} = (0 \ 0 \ 1)$$

Here, process (P_0) will be executed first because (R_1) resource will be executed.

$$\text{Availability of resource} = (0 \ 1 \ 1)$$

at this instance process (P_0) will be executed.

$$\text{Availability of resource} = (1 \ 1 \ 2)$$

Here;

process (P_1) will be executed

$$\text{Availability of resource} = (2 \ 2 \ 2)$$

Now,

process (P_2) will be executed.

30/02/24

This system does not contain deadlock.

methods to handle deadlock :-

(i) deadlock avoidance [safety method]

(ii) deadlock prevention

(a) mutual exclusion

(b) no preemption

(c) hold and wait.

(d) circular wait

⇒ Any one process should be false or all the four process has to be false for deadlock prevention.

(iii) deadlock avoidance [Banker's algorithm]

(iv) deadlock recovery.

(a.) mutual exclusion :-

⇒ Mutual exclusion is the process by which a single process is allowed to access the resource, which are non-shareable and is present in OS.

⇒ This condition ensures that a resource cannot be simultaneously accessed by multiple processes.

(b) No preemption :-

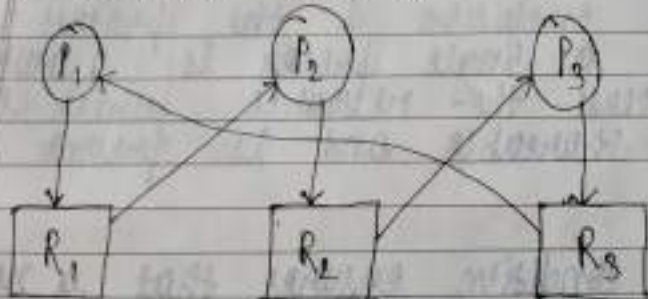
⇒ Resources cannot be preempted or forcibly taken away from a process. If a process is holding a resource and cannot complete its tasks due to waiting of another resource, it will not release its current resource, contributing to a potential deadlock.

(c) Hold and wait :-

⇒ The process must hold at least one resource while waiting to acquire additional resources.

⇒ This condition can lead to a situation where process hold some resources and wait indefinitely.

(d) Circular wait :-



⇒ This leads to a circular chain of one or more processes, each of which is waiting for a resource held by another process in the chain.

⇒ This circular waiting means no process in the cycle can make progress because another process blocks it.

31/08/24

definition

Deadlock Avoidance :- Banker's Algorithm

Process	Allocation			Max Need			Available			Remaining		
	A	B	C	A	B	C	A	B	C	A	B	C
P ₁	0	1	0	7	5	3	3	3	2	7	4	3
P ₂	2	0	0	3	2	2	5	3	2	1	2	2
P ₃	3	0	2	9	0	2	7	4	3	6	0	0
P ₄	2	1	1	4	2	2	7	4	5	2	1	1
P ₅	0	0	2	5	3	3	7	5	5	5	3	1
	7	2	5									

P₁ → P₄ → P₅ → P₃ → P₂

Given, A = 10
B = 5
C = 7

Remaining = Max need - Allocation

Given - Allocation sum = Available

Available = Available + Allocation

8)

Process	Allocation			Max need			Available			Remaining		
	A	B	C	A	B	C	A	B	C	A	B	C
P ₁	1	2	1	2	3	4	0	0	0	1	0	3
P ₂	2	0	1	2	1	3	5	4	3	0	1	2
P ₃	5	4	3	8	4	1	6	6	4	0	0	0
	8	6	5									

Given A = 5
B = 5
C = 5

• (P₃) → (P₁) → (P₂)

9)

Process	Allocation			MAX			Available			Remaining		
	E	F	G	E	F	G	E	F	G	E	F	G
P ₀	1	0	1	4	3	1	3	3	0	3	3	0
P ₁	1	1	2	2	1	4	4	3	1	1	0	2
P ₂	1	0	3	1	3	3	5	3	4	0	3	0
P ₃	2	0	0	5	4	1	6	4	6	3	4	1
	5	1	6									

• (P₀) → (P₂) → (P₁) → (P₃)

2/03/24

8) A system is having 3 processes each require 2 units of resources R. then, find the minimum no. of units of R such that no deadlock will occur.

(a) 3 (b) 5 (c) 6 (d) 4

let us take process P₁, P₂ & P₃

Given, Resource = 2 units

let us take a normal case:

process P₁, P₂ & P₃ will take 2 unit of resources to be executed

'R' = 3 × 2 = 6 → Normal case

case II

Now,

	P ₁	P ₂	P ₃	P ₁	P ₂	P ₃
case I	1	1	1	1	1	1
	1	1	1	1	1	1

Here P₁ will be executed and free up the resource then P₂ will take 2 resource and then P₃, but in this condition after the execution of process P₁, the process P₂ & P₃ can take 1 each resource, then occur a deadlock.

Now, $R = 5$ case II

Process	P_1	P_2	P_3	P_1	P_2	P_3
Case I	1	1	1	1	1	1

The process P_1 will be executed and then process P_2 and P_3 will be also allocated.

In another case if all allocate three resource but each process takes 1 resource each then deadlock will occur.

Now, $R = 4$ case III

Process	P_1	P_2	P_3	P_1	P_2	P_3
Case I	1	1	1	1	1	1

The process will be easily executed in each case and no deadlock situation will occur.

Resource for each process = (Minimum req + 1) No deadlock occurrence.

Maximum no. of resource allocation and then also deadlock will occur is: [Minimum requirements] i.e. = 8

Ex:

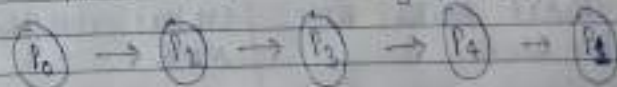
P_1	P_2	P_3
3	4	5
2	3	4

for minimum requirement of resource to prevent deadlock will be.

ex: $R = (2 + 3 + 4) + 1 = 10$

9)

Process	Allocation				Max. need				Available				Remaining				
	A	B	C	D	A	B	C	D	A	B	C	D	A	B	C	D	
P_0	0	0	1	2	0	0	1	2	1	5	2	0	0	0	0	0	0
P_1	1	0	0	0	1	7	5	0	1	5	3	2	0	7	5	0	0
P_2	1	3	5	4	2	3	3	4	2	3	3	6	1	0	0	2	0
P_3	0	6	3	2	0	6	5	2	2	14	0	2	0	0	2	0	0
P_4	0	0	1	4	0	6	5	6	2	14	12	0	0	6	4	2	0
	2	9	10	12					2	14	12						



* Types of threads:

(i) user level thread.

(ii) kernel level thread.

* Difference between user level and kernel level thread:

user level thread	kernel level thread
(i) It is implemented by user.	It is implemented by OS.
(ii) OS doesn't recognize user level thread.	kernel threads are recognized by OS.
(iii) Implementation of user threads is easy.	Implementation of kernel thread is complicated.
(iv) context switch time is less.	context switch time is more.
(v) context switch require no hardware support.	Hardware support is needed for context switch.
(vi) Ex - Java thread, POSIX threads.	Ex - Windows, Solaris.
(vii) If one user level thread perform blocking	If one kernel thread perform blocking operation then another

Hybrid level thread $\rightarrow$ combines both user and kernel level thread.

operation then within thread can continue
process is not blocked.

* Advantages of threads / Benefits:

- (i) Threads minimize the context switching time.
- (ii) All of threads provide concurrency with a process.
- (iii) Efficient communication.
- (iv) It is more convenient to create and context switch threads.
- (v) Threads allow utilization of multiprocessor architecture. It is a greater scale and efficiency.

* Scheduler :-

$\Rightarrow$ The main task is to select the jobs to be submitted into the system and to decide which process to run.

$\Rightarrow$ There are 3 types of scheduler. These are:
 1. Round Robin
 2. First In First Out
 3. Shortest Job First

Types of scheduler:

- (i) long term scheduler
- (ii) medium term scheduler
- (iii) short term scheduler

Differences between the type of scheduler:

long term	Medium term	short term
(i) It is job scheduler	(i) It is process swapping	(i) It is CPU scheduler
(ii) Speed is lesser than short term scheduler	(ii) Speed is in between short and long term scheduler	(ii) Speed is fastest among other two
(iii) It controls the degree of multi-programming	(iii) It reduces the degree of multi-programming	(iii) It provides lesser control over degree of multiprogramming
(iv) It is almost absent or minimal in time sharing system	(iv) It is a part of time sharing system	(iv) It is also minimal in time sharing system
(v) It selects process from pool and	(v) It can re-introduce the process into memory and	(v) It selects those processes which are ready to

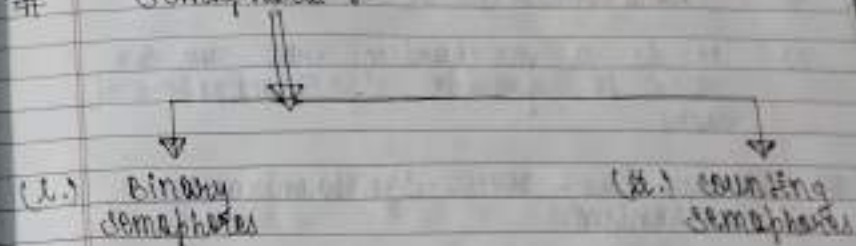
ready from pool
admission for
execution

execution can be
continued

can be
executed

* Through put - no. of processes that complete their execution for unit time.

Semaphores:-



(i) Binary semaphores:-

- ⇒ It is also known as mutex lock.
- ⇒ It can have only two values 0 & 1.
- ⇒ It's default value is initialized with 1
- ⇒ It is used to implement the solution of critical section problem with multi-processes

(ii) counting semaphores:-

Its value can range over an unrestricted domain.

It is used to control access to a resource that has multiple instances.

Characteristics of semaphores:-

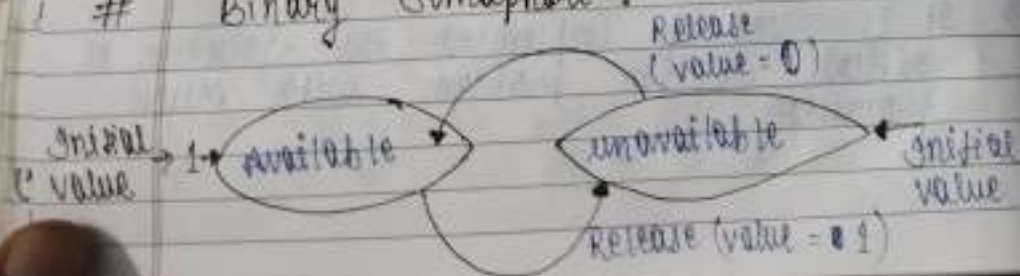
1) It is a mechanism that can be used to provide synchronization of tasks.

2) It is low-level synchronization mechanism.

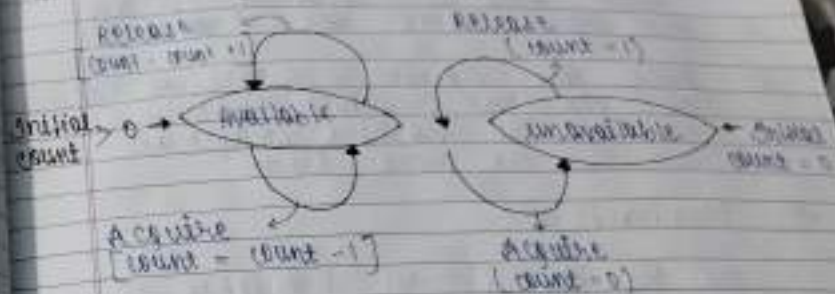
3) Semaphore will always hold a non-negative integer value.

4) Semaphore can be implemented using test operations and interrupts, which should be executed using file descriptors.

Binary Semaphore:-



counting semaphore:-



However, if the count is > 0 , the semaphore is created in the available state, and the number of tokens it has equal to its count.

Advantages of semaphores:-

1) It allows more than one thread to access the critical section.

2) Semaphore are machine independent.

3) They do not allow multiple-processes to enter the critical section.

4) They allow flexible management of resources.

⇒ as there is busy waiting in semaphore, there is never a wastage of process time and resources.

⇒ They are machine-independent code of the user, which should be run in the machine independent code of the machine-kernel.

Disadvantages :-

1) One of the biggest limitation of the semaphore is priority inversion.

2) The OS has to keep track of all codes to wait and signal semaphores.

3) Semaphore is prone to programmer error.

4) It is also not practical for large scale use as their use leads to loss of modularity.

Difference between counting and Binary semaphore :-

Counting semaphore

Binary semaphore

1) No mutual exclusion

1) Mutual exclusion

2) Any integer value as value only 0 and 1

3) more than one can

4) provide a set of

5) only one can

6) it has a mutual exclusion mechanism.

Linux OS :-

⇒ It was developed by Linus Torvalds in the year 1991.

⇒ It is a OS, which can be used by anybody and one can change the code according to their need.

⇒ When it was developed, its name was "freax".

Features of Linux :-

(i) Multiuser capability :-

⇒ Multiple users can access the same system resources like memory, hard disk, etc. but they have to use multiple different terminals to operate.

(iv) Multitasking :-

⇒ more than one function can be performed simultaneously by dividing the CPU time intelligently.

(v) Portability :-

⇒ portability doesn't mean it is smaller in size as it can be carried in pen drive or memory cards. It means that it supports different types of hardware.

(vi) Security :-

⇒ It provides security in three ways namely authenticating (by assigning passwords and login ID), authorization (by assigning permission to read, write and execute) and encryption (converts file into an unreadable format).

(vii) Live CD / USB :-

⇒ Almost all linux distros provide live CD / USB so that users can run / try it without installing it.

(viii) Graphical user interface :-

⇒ linux is command line based OS but it can be converted to GUI based by installing packages.

(ix) Support's customized keyboard :-

⇒ As it is used worldwide, hence supports different languages keyboards.

(x) Application support :-

⇒ It has its own software repository from where users can download and install many applications.

(xi) File system :-

⇒ provides hierarchical file system in which files and directories are arranged.

(xii) Open source :-

⇒ linux code is freely available to all and is a community based development project.

⇒ Some of the important types of kernel are :-

(i) Monolithic kernel [user signal and kernel signal both are implemented in same memory]

(ii) Micro kernel.

(iii) Exo kernel [can directly access hardware resource, developed by MIT]

(iv) Hybrid kernels.

(ii) System libraries :-

⇒ These libraries are specified as some special functions.

(iii) System utility programs :-

⇒ It is responsible for doing specialized level and individual activities.

(iv) Hardware layer :-

⇒ Linux OS contains a hardware layer that consists of several peripheral devices like CPU, HDD, and RAM.

v) Shell :-

⇒ It is an interface among the kernel and user.

It can take commands through the user and runs the functions of the kernel.

The shell is categorized into various shells.

(a) Graphical shells.

(b) Command-line shells.

#	Graphical shells	Command-line shells
(i)	Graphical line shells facilitate the graphical user-interface.	Command line shells facilitate the command line interface.
(ii)	The graphical user interface shell works slower as compared to command line interface shells.	(a) The command line user shell works comparatively faster than the graphical user interface shells.

Linux memory management :-

⇒ The subsystem of Linux is responsible for memory management.

⇒ It contains the implementation of demand paging and virtual memory.

05/05/24

- ⇒ It contains memory allocation of each space, programs and kernel internal structures.
- ⇒ Linux memory management is a complicated system with several configurable settings.
- ⇒ Almost every setting is available by the /proc filesystem and could be adjusted and queried with sysctl.

(i) The CPU can convert any virtual address into the physical address.

(ii) (i.) Huge pages

(iii) virtual memory primer.

(iv) Zones.

(v.) Page cache

(vi.) Nodes

(vii.) Anonymous memory

(viii.) OOM Killer

(ix.) compaction

(i.) Reclaim.

(ii.) Huge pages:-

⇒ The translation of addresses requires various memory accesses.

These memory accesses are very slow as compared to the speed of the CPU.

To ignore spending precious cycles on the processor on the translation of the address, CPUs manage the cache of these types of translations known as Translation Lookaside Buffer (TLB).

(ii.) virtual memory primer:-

The physical memory isn't contiguous necessarily.

It will make dealing with physical memory directly quite difficult to ignore this complexity a mechanism virtual memory was specified.

It permits to keep only required details inside the physical memory.

It gives a mechanism for controlled data sharing and protection between processes.

(iii) Zones :-

⇒ Linux combines memory pages into some zones according to the possible usage.

E.g. ZONE_HIGHMEM will include memory that isn't mapped permanently into the address space of the kernel.

(i) zone_DMA will include memory that could be used by various devices for DMA.

(ii) zone_Normal will include addressed pages normally.

(iv) Page cache :-

⇒ The common case to get data into memory is to read it through files as the physical memory is unstable.

⇒ The data will be put in the page cache to ignore expensive access of disk on the subsequent reads, whenever any file is read.

examples

(v) Nodes :-

⇒ Several multi-processor machines can be defined as the shared-memory architecture.

⇒ The memory is organized into chunks that include distinct access latencies helping on the "distance" through the protection in these types of systems.

(vi) Anonymous memory :-

⇒ These types of mappings are implicitly developed for heap and stack of the program, or by explicitly calls to the `mmap(2)` system call.

⇒ The anonymous mappings usually only specify the size of virtual memory that a program is permitted to access.

(vii) OOM killer :-

It is feasible that the kernel would not be able to reclaim sufficient memory and the loaded machine would be exhausted to proceed to implement.

(viii) compaction:-

- ⇒ As the system executes, various tasks allocate the free up the memory space and it becomes partitioned.
- ⇒ Memory compaction defines the positioning problems.

(ix) Reclaim:-

- ⇒ The pages that could be freed either due to they cache the details that existed elsewhere on a hard disk or due to they could be again swapped out to a hard disk, are known as reclaimable.

file management in linux:-

- ⇒ In linux, most of the operations are performed on files. To handle this linux has directories also known as folders which are maintained in a tree like structure.

⇒ linux has three types of files:-

(i) Regular files:-

- ⇒ It is the common file type in linux.
- ⇒ It includes files like, text files, images, binary files, etc.
- ⇒ Such files are created using touch command.

- ⇒ The regular files contains ASCII or human readable text, executable program binaries, program data and much more.

(ii) Directories:-

- ⇒ These are the files that store the list of file names and related information.

The root directory (/) is the base of the system, /home/ is the default location for user's home directories, /bin for essential user binaries, /usr - static text files, etc.

we could create new directories with mkdir command.

(iii) Special files:-

⇒ Represents a real physical device, such as a printer which is used for I/O operations.

* Note :-

⇒ In Unix system, there are two types of special files.

(i) character special files

(ii) block special files.

(i) # commands of files :-

(i.) Files listing :-

Syntax :-

⇒ ls command is used. $\$$ is Dollar

(ii.) creating files :-

⇒ touch command is used for creating a new file.

(i) # Syntax :-

⇒ $\$$ touch filename

(iii.) displaying file contents :-

⇒ cat command is used.

* Syntax :-

⇒ $\$$ cat filename

(iv.) copying a file :-

⇒ cp command is used

* Syntax :-

⇒ $\$$ cp source / filename destination /

(v.) moving a file :-

⇒ mv command is used

* Syntax :-

⇒ $\$$ mv source / filename destination /

(vi.) Renaming a file :-

⇒ mv command is used

* Syntax :-

⇒ $\$$ mv filename new - filename

(vii.) deleting a file :-

⇒ rm command is used.

Syntax:-

⇒ `ls filename`

Device Drivers in Linux:-

⇒ Drivers are used to help the hardware to interact with the OS.

In Linux, even the hardware devices are treated like ordinary files which makes it easier for the software to interact with the device drivers.

When a device is connected in the system, a device file is created in `/dev` directory.

Disk and Driver commands:-

(i.) `fdisk`:-

⇒ It stands for format disk.

⇒ This command is used to display the partitions on a disk and other details related to the file system.

⇒ Syntax:- `sudo fdisk -l`

(ii.) `lsdisk`:-

⇒ This command displays the partitions on the disk, the size of each partition in MB, etc.

(iii.) `parted`:-

⇒ This command helps list and modify the partitions of the disk.

Syntax:-

⇒ `sudo parted -l`

(iv.) `df`:-

⇒ Displays the details of the file system. Using `grep` we can filter real hard disk files.

Syntax:-

⇒ `df -h | grep ^/dev`

(v.) `lsblk`:-

⇒ List details about the block devices.

Syntax:-

⇒ `lsblk`

(vi) inxi :-

⇒ Gives details about the hardware components in the file system.

Syntax :-

⇒ `inxi -j -xx`.

Memory management in OS :-

⇒ Degree of multiprogramming means that the number of processes can enter the RAM for the execution.

Logical and physical address :-

(a) Logical Address :-

⇒ It is generated by CPU while a program is running.

⇒ The logical address is virtual address as it does not exist physically. Therefore, it is also known as virtual address.

⇒ This address is used as a reference to access the physical memory location.

(b) Physical Address :-

⇒ The term logical address space is used for the set of all logical addresses generated by a program's execution.

⇒ The hardware device called Memory Management Unit (MMU) is used for mapping logical address to its corresponding physical address.

(b) Physical Address :-

⇒ It identifies a physical location of requested data in a memory.

⇒ The user never directly deals with the physical address but can access by its corresponding logical address.

⇒ The user program generates the logical address and thinks that the program is in this logical address but the program needs physical memory for its execution.

⇒ Therefore, the logical address must be mapped to the physical address by MMU before they are used.

⇒ The term physical address space is used for all physical address.

Primary memory used for allocation and
deallocation of memory
i.e. a program running and processing it



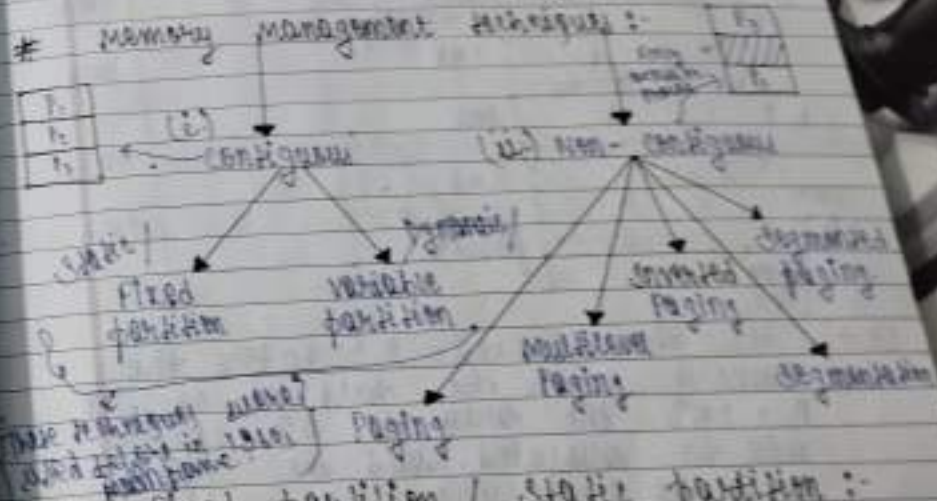
Difference between logical address and physical address:-

Parameter	Logical address	Physical address
Generation	Generated by CPU	Location in a memory
Address space	Logical address space is set of all logical addresses generated by CPU in reference to a program.	Physical address is set of all physical addresses mapped to the corresponding logical address.
Visibility	User can view the logical address of a program.	User can never view physical address of program.
Generation	Generated by the CPU.	Computed by MMU.

Process of multiprogramming or CPU utilization

The user can see the logical address to view the physical address.

Memory management techniques:-



Fixed partition / static partition:-

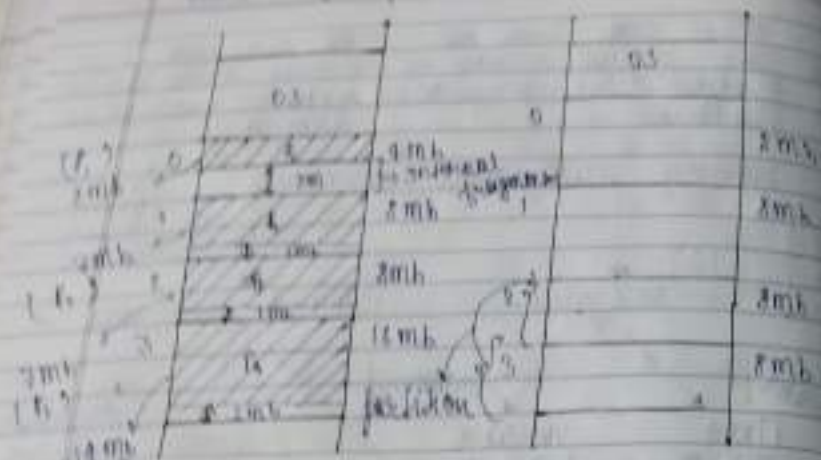
⇒ No. of partitions are fixed.

⇒ Size of the partition may or may not be same.

⇒ contiguous allocation so, spanning is not allowed.

Register takes instruction which is going to be executed.

is the id used to hold the address of that process which is frequently used by the user.



Ex -> let the process P_1 enter the memory with the help of OS, let $P_1 = 1\text{mb}$. the area of 0th position will be allocated. and the space left at 0th position cannot be used by any other process because we have partitioned the memory.

Here, process P_2 enters the memory, let $P_2 = 2\text{mb}$, the area allocated at 1st position. and vice versa.

If process P_5 want to allocate memory of size = 5mb it could not because of partition of the memory.

Disadvantages of fixed partitioning:-

- (i.) Internal fragmentation
process in which the memory space is left.
- (ii.) Limit in process size.
- (iii.) Limitation on degree of multiprogramming.
- (iv.) External fragmentation
process whose size can be fit in the memory but space are left in the partitioned data that cannot be utilized.

Advantages of fixed partitioning:-

- (i.) Implementation is easy.
- (ii.) Allocation and deallocation is easy.
- (iii.) memory is allocated before.
- (iv.) used mainframe computer in 1960s.
- (v.) memory is in the form of partitioned.

variable partitioning / dynamic partitioning
 → In this type of process the memory is allocated at the time of the processing.

Advantages of variable partitioning:-

let	OS	
$P_1 = 3\text{mb}$	P_1	3mb
$P_2 = 2\text{mb}$	P_2	2mb
$P_3 = 4\text{mb}$	P_3	4mb
$P_4 = 7\text{mb}$	P_4	7mb

(i.) No internal fragmentation

(ii.) No limitation on number of processes

(iii.) No limitation on process size.

Disadvantages of variable partitioning:-

(i.) External fragmentation

(ii.) Allocation / Deallocation is complex.

compaction:-

⇒ It can be done to allocate a process of greater size in a partition.

$P_1 = 8\text{mb}$	hole		2mb
	P_1	→	P_1
	P_2		P_2
	hole		P_3
	P_3		

OS	OS	OS
P_1	hole	P_5 10mb
P_2	→ P_2	→
P_3	hole	P_3
P_4	P_4	P_4

process (P_1) & (P_3) executed then it arises hole, that can be removed with the help of compaction.

Memory Allocation techniques :-

(i) First-fit :-

⇒ allocate the first hole that is big enough.



(ii) Next-fit :-

⇒ same as first fit but start search always from last allocated hole.

⇒ It will search the hole after the last allocation was done.

(iii) Best fit :-

⇒ Allocate the smallest hole that is big enough.

(iv) Worst fit :-

⇒ Allocate the largest hole.

Q. Request from processes are 300KB, 25KB, 125KB, 50KB, respectively and in the sequence, the above requirements could be satisfied with.

(A) Best fit but not first fit.

(B) first fit but not best fit.

(C) Both.

(D) None.

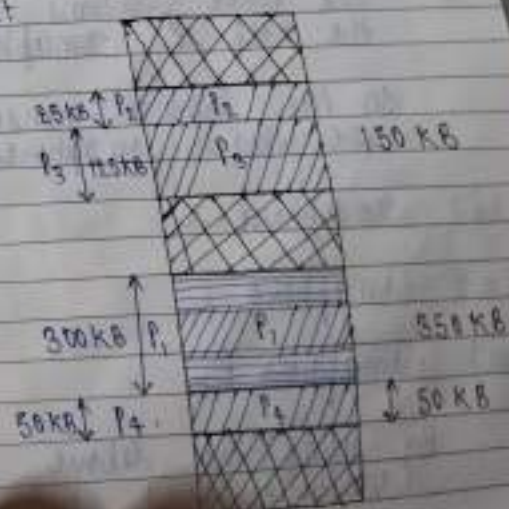
⇒ for first fit

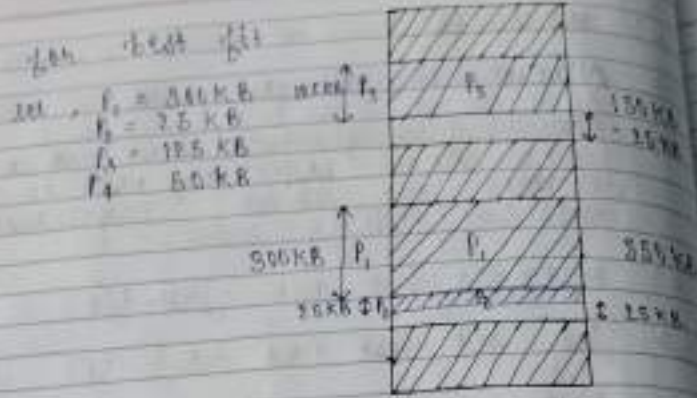
Let $P_1 = 300KB$

$P_2 = 25KB$

$P_3 = 125KB$

$P_4 = 50KB$





06/08/24

solutions for producer and consumer problem:-

⇒ The three problems can be solved with the help of semaphores.

In the producer-consumer problem, we use three semaphores variables:-

(i) Semaphore S:

(ii) Semaphore E:

(iii) Semaphore F:

By using the above three semaphores variables and using the wait()

and signal() function, we can solve the producer-consumer problem. The wait() function decreases the semaphore variable by 1 and the signal() function increases the semaphore variable by 1. So, let us see how.

(i) Semaphore S:-

⇒ This semaphore variable is used to achieve mutual exclusion between processes.

⇒ By using this variable, either producer or consumer will be allowed to use or access the shared buffer at a particular time. This variable is set to 1 initially.

(ii) Semaphore E:-

⇒ This semaphore variable is used to define the empty space in the buffer. Initially, it is set to the whole space of the buffer, i.e. "n" because the buffer is initially empty.

(iii) Semaphore F:-

⇒ This semaphore variable is used to

define the space that is filled by the producer. Initially it is set to 0, because there is no space filled by the producer initially.

Non-contiguous memory allocation:-
linked list can solve this problem if you have time.

Paging:-

⇒ Paging is a storage mechanism that allows OS to retrieve processes from the secondary storage into the main memory in the form of pages.

⇒ In the paging method, the main memory is divided into small fixed size blocks of physical memory, which is called frames.

⇒ The size of a frame should be kept the same as that of a page to have maximum utilization of main memory and to avoid external fragmentation.

⇒ Paging is used for faster access to data, and it is a logical concept.

Ex- If the main memory size is 10MB and frame size is 1MB, then the main memory will be divided into collection of 10 frames of 1MB each.

There are 3 separate processes in the system that is P1, P2, P3 and P4 of 8KB each. Here, all the processes are divided into pages of 1KB each so that OS can store one page in one frame.

At the beginning of the process, all the frames remaining empty in that all the pages of the processes will get stored in a contiguous way.



[collection of frames 'paging']

Paging protection:

⇒ The paging process should be protected by using the concept of instruction by an additional bit called valid / invalid bit.

⇒ Paging memory protection in paging is achieved by associating protection bits with each page.

⇒ These bits are associated with each page table entry and specify protection on the corresponding page.

(a.) Advantages of paging:-

(i.) Easy to use memory management algorithms.

(ii.) No need for external fragmentation.

(iii.) Swapping is easy between equal-sized pages and pages frames.

(b.) Disadvantages of paging:-

(i.) May cause internal fragmentation.

(ii.) Complex memory management algorithm.

Advantages of paging:-

(i.) Page tables consume additional memory.

(ii.) Multi-level paging may lead to memory reference overhead.

(iii.) Hardware support for paging:-

(i.) A set of dedicated registers holding base addresses of frames.

(ii.) In memory page table with a page table base register (PTBR).

⇒ Fixed sized partition:- Page table.

⇒ In this, the memory is divided into fixed-sized blocks and each block contains exactly one process. But the fixed sized partition will divide the number of processes.

Variable-size partition:-

⇒ In the variable size partition method, the OS maintains a table that contains the information about all memory parts that are occupied by the processes and all memory parts that are still available for the process.

Fragmentation :-

→ Fragmentation can either be external or internal fragmentation.

(i) External fragmentation :-

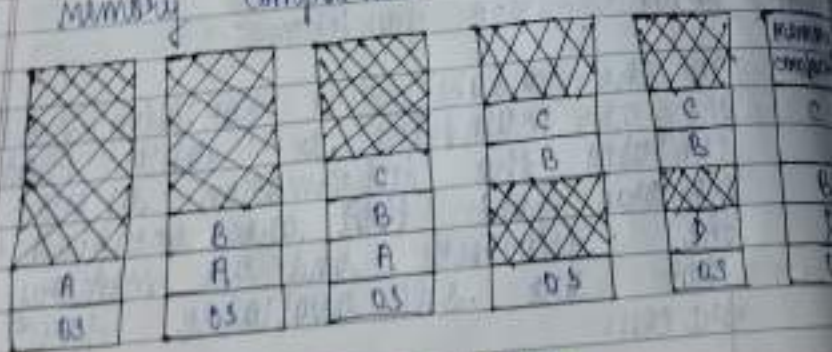
→ when the free memory blocks available in memory are too small and were non-contiguous.

(ii) Internal fragmentation :-

→ It occurs when the process does not fully utilize the memory allocated to it.

solution for the problem of external fragmentation :-

→ Memory compaction



Memory compaction

Advantages and Disadvantages of memory compaction:

→ The main advantage of contiguous memory allocation is memory access and predictability.

• As the memory is allocated to a file or a process, keeping in mind that it will grow during the run but until a piece of file needs more the main block allocated to it remains unused and then cannot be allocated to the other process leading to wastage of memory.

• In case, the process on the disk grows beyond the expectation, i.e. beyond the allocated memory block, then it will abort the "message" no disk space" leading to inflexibility.

→ The advantage of contiguous memory allocation is it increases the processing speed.

• As the OS shifts the buffered I/O and reads the process memory blocks consecutively it reduces the head movements. This helps in processing.

Definition of Banker's algorithm:

The Banker's algorithm is a resource allocation and deadlock avoidance algorithm that tests for safety by simulating the allocation for predetermined maximum possible amounts of all resources, then makes an "S-state" check to test for possible activity, before deciding whether the allocation should be allowed to continue.

Difference between Internal fragmentation and External fragmentation:

Internal fragmentation	External fragmentation
(i) In internal fragmentation, fixed-sized memory blocks square measure appointed to process.	(i) In external fragmentation, variable-sized memory blocks square measure appointed to the method.
(ii) It happens when the method or process is smaller than the memory.	(ii) It happens when the method or process is removed.
(iii) The solution of internal fragmentation is the best-fit block.	(iii) The solution of external fragmentation is paging and compaction.

(iv) Internal fragmentation occurs when memory is divided into fixed-sized partitions.	(iv) External fragmentation occurs when memory is divided into variable size partitions based on the size of process.
(v) The difference between memory allocated and scheduled space or memory is called internal fragmentation.	(v) The unused space formed between non contiguous memory fragments are too small to serve a new process, which is called external fragmentation.
(vi) Internal fragmentation occurs with paging and fixed partitioning.	(vi) External fragmentation occurs with dynamic partitioning and segmentation.
(vii) It occurs on the allocation of a process to a partition greater than the process's requirement. the leftover space causes degradation of system performance.	(vii) It occurs on the allocation of the process to a partition greater than the process's requirement. the leftover space causes degradation of system performance.
(viii) It occurs in worst fit memory allocation method.	(viii) It occurs in best fit and first fit memory allocation method.

No. of page = No. of frames
size

Paging concept :-

Ex →
drawn

Process size = 4 B

Page size = 2 B

No. of Page / Process

Process size / Page size

4 B / 2 B = 2

Main memory size = 16 B

Frame size = 2 B

Number of frames = main memory size / frame size
= 16 B / 2 B

Page no. = 8 frames (0 to 7)

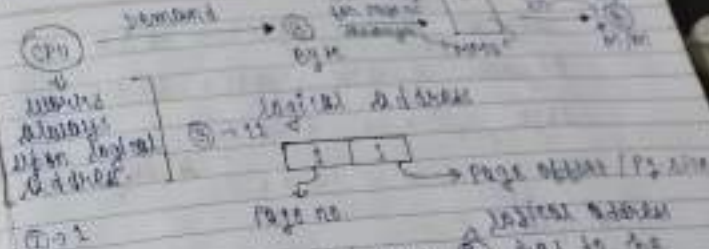


Page table of P₁ :-

0	f2
1	f4

frame 2
frame 4

Major - Main memory
minor - secondary storage unit



Binary number :-

Byte (10)	bit
1	1
2	10
3	11
4	100
5	101
6	110
7	111
8	1000
9	1001
10	1010
11	1011
12	1100
13	1101
14	1110
15	1111
16	10000
17	10001
18	10010
19	10011
20	10100

Now 5 has to be converted into physical address
10 → 10000000
+ 3 → 10000001
+ 1 → 100000010
P. address = 100000010
frame number = 100000010

collectively 1001 can be written as 9 in binary number

Hence we have fetched the value of 8 by the help of logical and physical address.

No. of entries in page table = No. of pages in a process

MSB $\rightarrow$ Most significant bits.
LSB $\rightarrow$ Least significant bits.

(L.A.S)

Q) Logical address space = 4 GB

Physical address space = 64 MB

Page size = 4 KB

Frame size = 4 KB } Always same

To find :-

No. of pages = 2^{20}

No. of frames = 2^{16}

No. of entries in page table = 2^{20}

Size of page table = $2^{20} \times 14$ bits

$2^1 = 2$

$2^2 = 4$

$2^3 = 8$

$2^4 = 16$

$2^5 = 32$

$2^6 = 64$

$2^7 = 128$

$2^8 = 256$

$2^9 = 512$

$2^{10} = 1K$

$2^{20} = 1M$

$2^{26} = 1G$

$2^{30} = 1T$

L.A.S = 4 GB

= $2^9 \times 2^{30}$

= 2^{39} = 82 bits

P.A.S = 64 MB

= $2^6 \times 2^{20}$

= 2^{26} = 26 bits

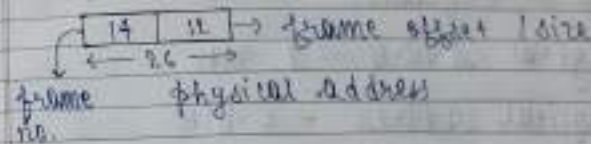
P.S = 4 KB

= $2^2 \times 2^{10}$

= 2^{12} = 12 bits

F.S = 4 KB

= $2^2 \times 2^{10} = 2^{12} = 12$ bits



$\therefore$ [No. of entries in page table = No. of pages in a process]

Size of page table :-

		14 frame no. in bits
0	P ₀	0000000000000000
1	P ₁	0000000000000001
2	P ₂	0000000000000010
	P ₃	0000000000000011
	P ₄	0000000000000100
	P ₅	0000000000000101
	P ₆	0000000000000110
	P ₇	0000000000000111
	P ₈	0000000000001000
	P ₉	0000000000001001
	P ₁₀	0000000000001010
	P ₁₁	0000000000001011
	P ₁₂	0000000000001100
	P ₁₃	0000000000001101
	P ₁₄	0000000000001110
	P ₁₅	0000000000001111
	P ₁₆	0000000000010000
	P ₁₇	0000000000010001
	P ₁₈	0000000000010010
	P ₁₉	0000000000010011
	P ₂₀	0000000000010100
	P ₂₁	0000000000010101
	P ₂₂	0000000000010110
	P ₂₃	0000000000010111
	P ₂₄	0000000000011000
	P ₂₅	0000000000011001
	P ₂₆	0000000000011010
	P ₂₇	0000000000011011
	P ₂₈	0000000000011100
	P ₂₉	0000000000011101
	P ₃₀	0000000000011110
	P ₃₁	0000000000011111

$\Rightarrow$ The size of bit can have various more number of data such as page is protected, Modified or not, etc.

default { 1 word = 1 byte }
if not mention

Q. consider a system which has
logical address (LA) = 7 bits, physical
address = 6 bits, page size = 8 words.
then calculate no. of pages &
no. of frames.

Given

logical address = 7 bits

physical address = 6 bits

page size = 8 words = 2^3 = 3 bits

logical address

page no. $\leftarrow \begin{array}{|c|c|} \hline 4 & 3 \\ \hline \end{array} \rightarrow$ page offset
 $\leftarrow 7 \rightarrow$

Physical Address

frame no. $\leftarrow \begin{array}{|c|c|} \hline 9 & 3 \\ \hline \end{array} \rightarrow$ frame offset
 $\leftarrow 6 \rightarrow$

No. of pages = $2^7 = 16$

No. of frames = $2^6 = 8$

08/08/24

Page table entry :-

Frame No.	valid (1) / invalid (0)	protection (R, W, X)	Reference (0/1)	cacheing (enable / disable)	dirty
mandatory field		optional field			

(i) valid (1) / invalid (0) :-

$\Rightarrow$ It is used to search in the memory
about the page, that is stored or
not stored in the memory.

(ii) Protection :-

$\Rightarrow$ The data can be fetched using Read,
write and execute method for
protecting it.

(iii) Reference :-

$\Rightarrow$ LRU Method, i.e. ; least recently used
method. used for swapping values
one by one
for multiple time

Replacing value, one by one = 1 $\rightarrow$ fetching
using data for 1st time = 0 $\rightarrow$ getting

(v.) caching :- (CPU)

⇒ caching helps in retrieving data at a faster rate for execution of data.

⇒ Enable and disable has to be done.

(vi.) Dirty / Modify :-

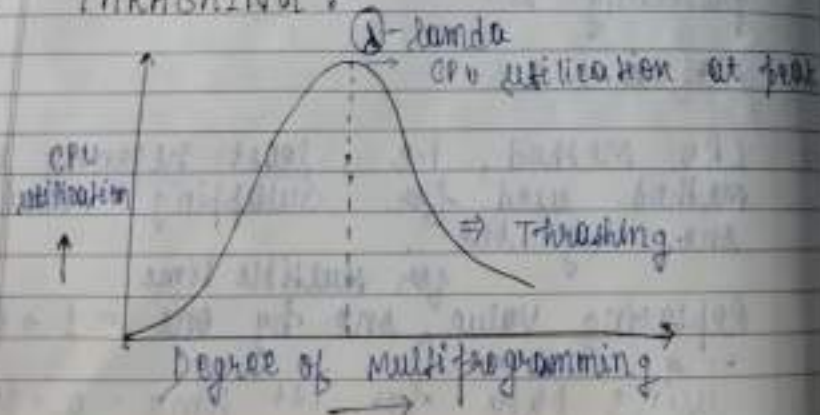
⇒ used for modifying the data.

Ⓐ Modified data = 1

Ⓑ unmodified data = 0.

⇒ the value is changed in the main memory and the value is changed in the HDD with the help of modified data = 1.

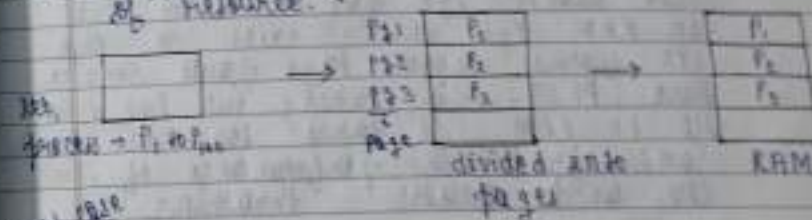
THRASHING :-



Thrashing is directly linked with degree of multiprogramming.

We use the concept of paging for the utilization of CPU at a higher rate.

Here, the process are converted into various different pages for better utilization of resources.



Best case

For maximum utilization of process in CPU, we use to accommodate RAM with the paging. i.e. a process doesn't enter entirely in the RAM its some 10 to 20 percentage of data is entered.

Worst case

When we continuously put the process too fast, i.e. degree of multiprogramming then at a point (i) it start degradation because of the following reason.

$P_1 \rightarrow \text{Process}$
 $P_2 \rightarrow \text{Page}$

P_1	P_{21}
P_1	P_{21}
P_1	P_{21}
P_1	P_{21}
P_1	P_{21}
P_1	P_{21}
P_1	P_{21}
P_1	P_{21}
P_1	P_{21}
P_1	P_{21}

RAM

When the process P_1 was executing, the CPU wants P_{22} for execution, but in RAM P_{22} doesn't exist so, the CPU waits upon Page fault service time [PFS], for taking out P_{22} from HD to RAM. It takes too much time and degrades the performance of the CPU and creates thrashing.

To Avoid thrashing :-

(i) By increasing the size of the RAM.

(ii) To tell the long term scheduler to degrade / slow your performance because it is responsible for degree of multiprogramming.

Segmentation :-

It is a process in which the process is segmented on the basis of user needs, i.e. every section is converted into the section.

In paging process, it is paged randomly.

main()	P_1	main()	P_1
Add()	P_2	Add()	P_2
Reverse()	P_3	Reverse()	P_3

Segmentation

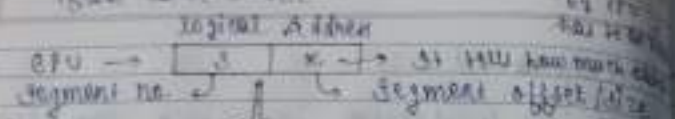
Each Segment

main()	Add()
S_1	S_2
Sub()	Mod()
S_3	S_4
Sort()	Power()
S_5	S_6

RAM.

It will be executed by the CPU.

every segment size is vary from 64K to other.



Program 1

for 1: $x = 400 \leq \text{Size}$
yes.
It will be executed by CPU.

Program 2

for 0: $x = 100 \leq \text{Size}$
yes.
It will be executed by CPU.

Program 3

for 5: $x = 400 \leq \text{Size}$
no.
It will go trap.
no execution by CPU.

Overlay:

- ⇒ It is used when the program size is bigger than the memory size (secondary memory).
- ⇒ It is usually used in embedded system, such as ECU engine control unit, washing machine, etc.
- ⇒ It works as a virtual memory.

Q) Consider a two pass assembler Pass 1: 80 KB, Pass 2: 90 KB, Symbol table: 30 KB, common routine: 20 KB. At a time only one pass is in use what is min partition size required if overlay driver is 10 KB size.

80 KB
80 KB
80 KB
20 KB
+ 10 KB
= 280 KB

If available then no overlay is used.

Thus much storage is required.

PASS:1	PASS:2
80 KB	80 KB
80 KB	80 KB
80 KB	80 KB
10 KB	10 KB
150 KB	140 KB

hence any one pass can only use
so the minimum partition size
will be

$$\text{Pass:2} = 150 \text{ KB} \dots$$

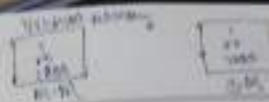
Virtual memory:

⇒ It is a space where large programs
can store themselves in form of
pages while their execution and only
the required pages or portions of
processes are loaded into the main
memory.

⇒ This technique is very useful as large
virtual memory is provided for such
programs when a very small physical
memory is there.

Benefits of having virtual memory:

(i.) Large programs can be written as
virtual space available is huge as
compared to physical memory.



(ii.) Less I/O required, leads to faster and
easy swapping of processes.

(iii.) More physical memory available as programs
are stored in virtual memory, so they
occupy very less space in actual
physical memory.

Demand paging:

⇒ The basic idea behind demand paging is
that when a process is swapped in,
its pages are not swapped in all at
once. Rather they are swapped in
only when the process needs them
(on demand).

⇒ This is termed as lazy swapper, although
a pager is a more accurate term.

⇒ Initially only those pages are loaded
which will be required the process
immediately.



→ The page that are not moved into memory are marked as invalid in the page table for an invalid entry the rest of the table is empty.

→ In case of pages that are loaded in the memory they are marked as valid along with the information about where to find the swapped outpage.

④ When the process requires any of the page that is not loaded into the memory, a page fault trap is triggered and following steps are followed:

(i) The memory address which is requested by the process is first checked, to verify the request made by the process.

(ii) If it is found to be invalid, the process is terminated.

(iii) In case the request by the process is valid, a free frame is located, possibly from a free frames list, where the required page will be moved.

(iv) A new operation is scheduled to move the necessary page from disk to the specified memory location.

(v) When the I/O operation is complete the processes page table is updated with the new frame number and the invalid bit is changed to valid.

(vi) The instruction that caused the page fault must be now be restarted from the beginning.

⇒ There are cases when no pages are loaded into the memory initially, pages are only loaded when demanded by the process by generating page faults, this is called pure demand paging.

⇒ The only major issue with demand paging is, after a new page is loaded, the process starts execution from the beginning.

⇒ It is not a big issue for small programs, but for larger programs it affects performance drastically.

Page Replacement :

→ This allow us to get more no. of processes into the memory at the same time. But what happens when the process request for more pages and no free memory is available to bring them in.

The following steps can be taken to deal with this problem:-

- (2) put the process in the wait queue until any other process finishes its execution thereby freeing frames.

- (ii) If, remove some other process completely from the memory to free frames.

- (iii) Oh, find some pages that are being used right now, move them to the disk to get free frames. This technique is called page replacement and is most commonly used. We have some great algorithms to carry on page replacement efficiently.

There are three types of page replacement algorithm.

- (ii) FIFO

- ### (3.9) Optimal Page Replacement

- (iii) Least Recently used (LRU)

- 123 FIED :-

→ A very simple way of page replacement is FIFO.

⇒ It is an effective way of page replacement but can be used for small systems.

69 consider the following reference string

7, 0, 1, 2, 0, 3, 0, 4, 2, 3, 0, 3, 2, 1
2, 0, 1, 3, 0, 1

Assume there are three frames, apply FIFO replacement algorithm to the reference string above and find out how many page faults are produced.

[illegible]

Let $x = \text{page fault}$

Hit = page^c is there

page fault = 15

page hit = 5

$$\text{page fault ratio} = \frac{15}{20} \times 100 \\ = 75\%$$

$$\text{page hit ratio} = \frac{5}{20} \times 100 \\ = 25\%$$

Belady's Anomaly in FIFO :-

⇒ It is the process in which the frame size is increased by 1 then the no. of hits degrades degrades and page fault ↑ increases. for FIFO only.

9) Reference string :-

1, 2, 3, 4, 1, 2, 5, 1, 2, 3, 4, 5

f ₃			3	3	3	2	2	2	2	4	4				
f ₂		2	2	2	1	1	1	1	5	3	3				
f ₁	1	1	1	4	4	4	5	5	5	5	5				
	*	*	*	*	*	*	*	*	*	*	*	*	*	*	*

Page fault = *

Page hit = Hit

page fault = 5

page hit = 3

Belady's Anomaly

1, 2, 3, 4, 1, 2, 5, 1, 2, 3, 4, 5

f ₄					4	4	4	4	4	3	5	5			
f ₃					3	3	3	3	3	2	2	2			
f ₂		2	2	2	2	2	2	2	1	1	1	1			
f ₁	1	1	1	1	1	1	5	5	5	5	4	4			
	*	*	*	*	*	*	*	*	*	*	*	*	*	*	*

page fault = 10

page hit = 2

Optimal page replacement :-

⇒ The algorithm that has the lowest page fault rate of all algorithm and will never suffer from Belady's Anomaly. Such an algorithm does exist and has been called opt or min.

Replace the page that will not be used for the longest period of time.

09/09/24

Linux commands / open source technology

(i) ls command :-

⇒ Display information about files in the current directory.

(ii) Pwd command :-

⇒ displays the current working directory.

(iii) mkdir :-

⇒ creates the directory.

(iv) cd :-

⇒ To navigate between different folders.

(v) rmdir :-

⇒ Removes empty directories from the directory lists.

(vi) cp :-

⇒ copy files from one directory to another.

(vii) mv :-

⇒ Rename and replace the files.

(viii) rm :-

⇒ delete files.

(ix) uname :-

⇒ command to get basic information about the OS.

(x) locate :-

⇒ find a file in the database.

(xi) touch :-

⇒ create empty files.

(xii) ln :-

⇒ create shortcuts to other files.

(xiii) cat :-

⇒ display file contents in terminal.

(xiv) clear :-

⇒ clear terminal.

(xv) ps :-

⇒ Display the process in terminal.

(xvi.) man :-

⇒ Access manual for all linux commands

(xvii.) grep :-

⇒ Search for a specific string in an output

(xviii.) echo :-

⇒ Print string or text in the terminal

(xix.) wget :-

⇒ Download files from the internet

(xx.) whoami :-

⇒ Displays the current users name

(xxi.) sort :-

⇒ Sort the file content

(xxii.) cal :-

⇒ view calendar in terminal

(xxiii.) whereis :-

⇒ view the exact location of any command typed after this command

(xxiv.) df :-

⇒ check the details of the file system

(xxv.) wc :-

⇒ check the lines, words count, and character in a file using different options

wc -w → show number of words

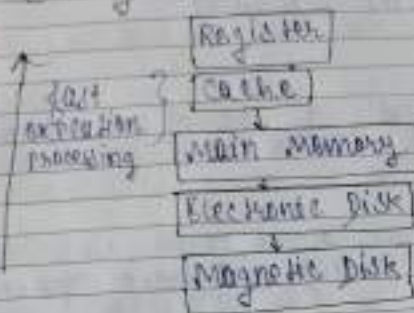
wc -l → show number of lines

wc -m → show number of characters present in a file

#	SR	TF	P ₀	P ₁	P ₂	P ₃	P ₄	P ₅	P ₆	P ₇	P ₈	P ₉	P ₁₀	P ₁₁	P ₁₂
	P	AT	BT	CT	TAT	W-T	R-T								
	P ₁	0	5	9	9	4	0								
	P ₂	1	3	4	3	0	0								
	P ₃	2	4	13	11	9	7								
	P ₄	4	1	5	1	0	0								

2/09/24

Storage Structure



12/09/24

Information System call:-

used for getid, system date & time.

Device System call:-

Read, write and open

Process System call:-

load, execute, fork

Communication System call:-

Pipe, create / delete

Security System call:-

chmod.

File system call:-

Read, write, open.

System calls in OS:-

(i) A system call is a way for a user program to interface with the OS.

(ii) A system call can be written in assembly language or a high level language like C or Pascal.

(iii) System calls are predefined functions that the OS may directly invoke if a high-level language is used.

Q:- What is a system call?

(i) A system call is a method for a computer program to request a service from the kernel of the OS.

(ii) A system call is a method of interacting with the OS via programs.

(iii) A system call is a request from computer software to an OS kernel.

(iv) The Application Program Interface (API) connects the OS functions to user programs.

Q) How are system calls made?

Some examples of how a system call varies from a user function are as follows:-

(i) A system call function may create and use kernel processes to execute the asynchronous processing.

(ii) A system call has greater authority than a standard subroutine.

(iii) A system call with kernel-mode privilege executes in the kernel protection domain.

(iv) System calls are not permitted to use shared libraries or any symbols that are not present in the kernel protection domain.

(v) The code and data for system calls are stored in global kernel memory.

Q) Why do you need system calls in OS?

The situation in which system call is needed are as follows:-

(i) It is must require when a program wants to create or delete a file.

(ii) Network connections require the system call to sending and receiving data packets.

(iii) If you want to read or write a file, you need to system call.

(iv) If you want to access hardware device, including a printer, scanner, you need a system call.

(v) System calls are used to create and manage new processes.

Q) How system call work?

(i) The Applications run in an area of memory known as user space.

(ii) When an application creates a system call, it must first obtain permission from the kernel.

(iii) It achieves this using an interrupt request, which pause the current process and transfer control to the kernel.

history -> the component which problem for and then component is added at the back.

- (iv) If the request is permitted, the kernel performs the request action, like creating or deleting a file.
- (v) When the application receives kernel's output, it resumes its procedure after the input is received.
- (vi) When the operation is finished, the kernel returns the results to the application and then moves data from kernel space to user space in memory.
- (vii) A simple system call may take few nano-seconds to provide the result, like retrieving the system date and time.
- (viii) A more complicated system call, such as connecting to a network device, may take a few seconds.
- (ix) Most OS launch kernel thread for each system call to avoid the bottlenecks.
- (x) Modern OS are multithreaded, which means they can handle various system calls at the same time.

Types of system calls:
 There are actually various system calls, i.e.

- (i) Windows has almost 200 system calls.
 - (ii) Linux has almost 300 system calls.
 - (iii) Unix has almost 300 system calls.
- The main five system calls are as follows:-

- (i) Process control
- (ii) File management
- (iii) Device management
- (iv) Information maintenance
- (v) Communication.

(i) Process control:-

- A) process control is the system call that is used to direct the process.
- B) some process control examples include

creating, load, abort, end, monitor
process, terminate the process, etc.

(ii) File management:

- a) File management is a system call that is used to handle the files.
- b) Some file management examples include creating files, delete files, open, close, read, write, etc.

(iii) Device management:

- a) Device management is a system call that is used to deal with devices.
- b) Some examples of device management include read, write, get device attributes, release device, etc.

(iv) Information maintenance:

- a) Information maintenance is a system call that is used to maintain information.
- b) Some examples are, including getting system data, set time or date, get time or date, set system data, etc.

communication:

communication is a system call that is used for communication.

Some examples are, create, delete communication connections, send, receive messages, etc.

Disk management:

What is a disk?

(i) It is a secondary storage device that is used to store data.

(ii) Disk provide a means to store a large amount of information for modern computer.

Ex → Hard disk, solid state drive, floppy disk.

Define disk structure?

(i) A disk is usually divided into tracks, cylinder and sectors.

(ii) Hard disks drives are organised as a concentric stack of disks or "platters".